

第 36 章：Exception Odds and Ends (异常的边角细节)

原书：《Learning Python》(6th Edition)，作者 Mark Lutz 本章地位：第 7 部分 (Exceptions and Tools, 异常与工具) 的收官章——用"边角话题 + 常见用法 + 设计概念"把异常主题补全，随后进入本部分的陷阱与练习。同时，这一章也为全书"核心语言"部分画上句号：从本章之后，你将正式告别"Python 语言入门者"身份，书中给出了一份从语言新手过渡到应用开发者的工具地图 (文档、测试、剖析、调试、打包、优化、虚拟环境等)，然后才进入全书最后一部分"高级主题"。

36.1 开篇引言

英文原文

This chapter rounds out this part of the book with the usual collection of stray topics, common-usage examples, and design concepts, followed by this part's gotchas and exercises. Because this chapter also closes out the fundamentals portion of the book at large, it includes a brief overview of concepts and developme

nt tools to help as you make the transition from Python language beginner to Python application developer.

中文翻译

本章以惯常的"零散话题 + 常见用法示例 + 设计概念"合集勾画完本书这一部分的轮廓，随后是本部分的陷阱总结 (gotchas) 与练习。由于这一章同时为全书的基础知识部分画上句号，它还包含对若干概念与开发工具的简要总览，帮助你完成从 Python 语言初学者到 Python 应用开发者的转变。

深度理解

·核心概念：本章没有新语法，而是"收尾与整合"——把前面几章没展开的异常细节（嵌套、惯用模式、设计权衡）一次补齐，并追加一份开发者工具箱导览。它解决的实际问题是：语法学会了，但"什么时候用异常、怎么用好异常"还没有形成直觉。

·底层视角：注意关键词 gotchas（陷阱）——作者 20 多年教学经验告诉你，异常本身简单到不用学，难的是把握 except 子句的尺度（多具体、多宽泛）以及哪里该包 try、哪里不该包。这些是"设计问题"而非"语法问题"。

·设计思想：把"设计提示"放在全书核心部分的最后，是作者一贯的倒金字塔结构——先给工具，再谈艺术。正因为这是核心语言部分的最后一章，它同时承担了"毕业演讲"的功能，预告后续的高级主题部分。

·初学者误区：以为“会写 try/except 就算学会异常了”。实际情况恰恰相反——异常的优雅使用之道（用它跳转、用它当信号、用类组织异常层次）比语法本身重要得多。

36.2 Nesting Exception Handlers (嵌套异常处理器)

英文原文

Most of our examples so far have used only a single try to catch exceptions, but what happens if one try is physically nested inside another? For that matter, what does it mean if a try calls a function that runs another try? Technically, try statements can nest in terms of both syntax and the runtime control flow through your code. This was mentioned earlier in brief but merits clarification here. Both of these cases can be understood if you realize that Python stacks try statements at runtime. When an exception is raised, Python returns to the most recently entered try statement with a matching except clause. Because each try statement leaves a marker on the top of a LIFO stack, Python can jump back to earlier tries by inspecting the stacked markers. This nesting of active handlers is what we mean when we talk about propagating exceptions up to "higher" handlers—such handlers are simply try statements entered earlier in the program's execution flow. Figure 36-1 illustrates what occurs when try state

ments with except clauses nest at runtime. The amount of code that goes into a try block can be substantial, and it may contain function calls that invoke other code watching for the same exceptions. When an exception is eventually raised, Python jumps back to the most recently entered try statement that names that exception, runs that statement's except clause, and then resumes execution after that try. Figure 36-1. Nested try/except combinations

Once the exception is caught, its life is over—control does not jump back to all matching tries that name the exception; only the first (i.e., most recent) one is given the opportunity to handle it. In Figure 36-1, for instance, the raise statement in the function func2 sends control back to the handler in func1, and then the program continues within func1. By contrast, when try statements that contain only finally clauses are nested, each finally block is run in turn when an exception occurs—Python continues propagating the exception up to other tries, and eventually perhaps to the top-level default handler (the standard error-message printer). As Figure 36-2 illustrates, the finally clauses do not kill the exception—they just specify code to be run on the way out of each try during the exception propagation process. If there are many try/finally combos active when an exception occurs, they will all be run unless an except clause catches the exception somewhere along the way. Figure 36-2. Nested try/finally combinations

In oth

er words, where the program goes when an exception is raised depends entirely upon where it has been—it's a function of the runtime flow of control through the script, not just its syntax. The propagation of an exception essentially proceeds backward through time to try statements that have been entered but not yet exited. This propagation stops as soon as control is unwound to a matching except clause, but not as it passes through finally clauses on the way. The prior chapter's except clauses don't change this story—if they consume every exception in the raised group, the aggregate exception ends in the try as usual. As we saw, unmatched excepts are reraised after the except clauses have their chance and are propagated on to other try statements or the top-level handler, but this is not fundamentally different from exceptions unmatched by an except.

中文翻译

截至目前我们的大多数示例都只用单个 try 来捕获异常，但如果一个 try 在字面上嵌套在另一个 try 里面，会发生什么？更进一步，如果一个 try 调用了一个运行着另一个 try 的函数，又意味着什么？从技术上讲，try 语句既可以按语法嵌套，也可以在运行时通过代码的控制流嵌套。之前只是简单提过这一点，此处值得澄清。只要认识到 Python 在运行时把 try 语句压入堆栈，这两种情况就都能理解了。当异常被抛出 (raise) 时，Python 会回溯到最近进入的、带有匹配 except 子句的那个 try 语句。由于每个 try 语句都会在

LIFO（后进先出）栈顶留一个标记，Python 可以通过检查这些堆叠的标记跳回到更早的 try。这种“活动处理器嵌套”正是我们常说的把异常向上传播到“更高层”处理器的含义——所谓更高层处理器，不过是在程序执行流中更早进入的 try 语句而已。图 36-1 展示了带 except 子句的 try 语句在运行时嵌套时会发生什么。一个 try 块中装入的代码量可能相当可观，内部还可能包含对“也在监视同一异常”的其他代码的函数调用。当异常最终被抛出时，Python 会跳回到最近进入的、点名了该异常的那个 try 语句，执行它的 except 子句，然后从这个 try 之后继续往下执行。图 36-1. 嵌套的 try/except 组合一旦异常被捕获，它的生命就结束了一——控制权不会跳回所有点名该异常、且与之匹配的 try；只有第一个（也就是最近的）那个才有机会处理它。例如在图 36-1 中，函数 func2 里的 raise 语句把控制权交还给 func1 中的处理器，之后程序便在 func1 内继续执行。相反，当只含 finally 子句的 try 语句嵌套时，异常发生时每一个 finally 块都会依次执行——Python 继续把异常向上传播给其他 try，最终也许传到顶层默认处理器（也就是标准错误信息打印器）。如图 36-2 所示，finally 子句并不会“杀死”异常——它们只是在异常传播过程中、离开每个 try 时规定要运行的代码。如果异常发生时有很多活跃的 try/finally 组合，除非中途某个 except 子句把异常捕获了，否则它们全部都会被运行。图 36-2. 嵌套的 try/finally 组合换句话说，异常抛出时程序去哪里，完全取决于程序去过哪里——它是脚本运行时控制流的函数，而不只是它的语法。异常的传播本质上是在“沿时间倒流”，回溯到那些已经进入、但尚未退出的 try 语句。一旦控制卷绕（unwind）到一个匹配的 except 子句，传播就停止了；但沿途穿过 finally 子

句时并不会停下。上一章的 `except` 子句并不会改变这个故事——如果它们消耗掉了抛出的异常组 (`exception group`) 里的每一个异常，那么这个聚合异常就照常终结在那个 `try` 里。正如我们所看到的，未被匹配的 `except` 在 `except` 子句处理完之后会被重新抛出，并继续传播到其他 `try` 语句或顶层处理器；但从根本上看，这与被 `except` 漏掉的异常并无本质差别。

深度理解

·核心概念：异常的行为是运行时控制流的函数，不是源代码"长得像什么"的函数。同一个异常被谁捕获，取决于此刻活跃的 `try` 链表。可以类比现实中的"层层上报"：你在公司里层层递交出了问题，处理它的不一定是第一级领导，而是最近一个愿意且有权管这件事的人；如果没人管，就一路报到 CEO（顶层处理器），CEO 处理不了就直接"打印错误然后终止"。

·底层实现：CPython 为每个 `try` 在求值栈 (`eval stack`) 或专门的异常处理栈上留下标记。抛出异常时，解释器从当前帧里保存的 `handler` 列表出发、按 LIFO 顺序查找 `except` 匹配。用 `dis` 反汇编可以看到：进入 `try` 后的代码块会被一条 `SETUPWITH/SETUPFINALLY` 之类的指令保护（3.11+ 起改为基于 JUMP 和异常表跳转表）。`finally` 与 `except` 的根本差别在实现上对称地体现出来：`except` 匹配到就消费异常、结束传播；`finally` 只是回调钩子，跑完代码接着把异常继续往上抛。

·设计原因：为什么让异常按 LIFO 传播、而不是"所有匹配的 `try` 都处理一遍"？因为这样语义最清晰、开销最小：

一个异常只有一个归宿。若异常同时被两层处理，必然产生"谁覆盖谁"的混乱。finally 则始终坚持"只要你经过我，就必须执行收尾"，这是它作为确定性清理工具的根基。

·实际问题：大型程序中"运行时嵌套"（A 调 B 调 C 引发的异常）远比语法嵌套常见。理解这一点后，调试时你就知道应该看调用栈而不是死盯某个文件里 try 的"位置"。

·初学者误区：误以为异常会被"所有"匹配的 try 各处理一次，或误认为 finally 会像 except 一样拦住异常。前者错在只处理一次；后者错在 finally 不消费异常——分不清这两点，就无法读懂任何带多层清理的程序。另外，except 处理的是"异常组"，即使它匹配了部分子异常，未匹配部分仍会继续传播，这与普通 except 的"一网打尽"语义不同。

补：异常的代价（本书未展开，属现代实践补充）——raise 有一个"建立回溯对象 + 逐帧展开栈"的过程，确实比普通分支跳转贵；但大多数 try 块在不抛异常时，CPython 3.11 之后实现了近乎零开销的异常处理（zero-cost exceptions：try 块不在热路径上做任何检查，只有真正抛异常时才查异常表把指令指针跳转回处理区）。因此，在 Python 里"多用 try 做防御"并不会显著拖慢热循环——这也反过来支持了本章后面"把整个函数调用包在一个 try 里"的建议。真正昂贵的是异常发生本身（回溯格式化与栈展开），所以不要用异常做常规业务逻辑的"返回值"。

36.3 Example: Control-Flow Nesting (示例：控制流嵌套)

英文原文

Let's turn to examples to make this nesting concept more concrete. The module file in Example 36-1 defines three functions: `action1` wraps a call to `action2` in a try handler, `action2` does likewise for a call to `action3`, and `action3` is coded to trigger a built-in `TypeError` exception (you can't add numbers and sequences). Example 36-1. `nestedexcnormal.py` Notice, though, that when `action3` triggers the exception, there will be two active try statements—the older one in `action1` and the newer one in `action2`. Python picks and runs just the most recent try with a matching except—which in this case is the try inside `action2`. In this demo, `action2` also manually reraises the `TypeError` with `raise` to trigger the try in `action1`, but the exception would otherwise die in `action2`: `$ python3 nestedexcnormal.py`

Inner try Outer try The same happens for an exception group, though the exception doesn't die until the entire group has been matched. In Example 36-2, for instance, `action2` picks off `IndexError`, `action1` consumes `TypeError`, and `SyntaxError` propagates to the top-level default handler (or an earlier matching try, if one had been run). Example 36-2. `nestedexcgroup.py`

When run, each function's try consumes exceptions it

matches, and the top-level handler prints an error message for the last remaining unmatched item in the group: \$ python3 nestedexcgroup.py Got IE Got TE + Exception Group Traceback (most recent call last): ... etc ... | ExceptionGroup: Nest (1 sub-exception) +-----
 ----- 1 ----- | SyntaxError: 3 +-----
 ----- Whether groups or individual exceptions are raised, the place where an exception winds up jumping to depends on the control flow through the program at runtime. Because of this, to know where you will go, you need to know where you've been. In other words, routing for exceptions nested at runtime is more a function of control flow than of statement syntax. That said, we can also nest exception handlers syntactically—an equivalent case we turn to next.

中文翻译

让我们来看示例，让这种嵌套概念更具体。示例 36-1 中的模块文件定义了三个函数：action1 在一个 try 处理器里包装了对 action2 的调用，action2 对 action3 的调用如法炮制，而 action3 被编写成故意触发内置的 TypeError 异常（你不能把数字和序列相加）。示例 36-1. nestedexcnormal.py 但要注意，当 action3 触发异常时，会有两个活跃的 try 语句——action1 里较早的那个，和 action2 里较新的那个。Python 只会挑选并运行最近的那个、且带匹配 except 的 try——本例中就是 action2 里面的 try。这个演示里，action2 还用 raise 手动重新抛出了 TypeError

，以触发 action1 里的 try；但如果没有这一步，异常本会在 action2 处就此消亡：\$ python3 nestedexcnormal.py Inner try # 内层捕获Outer try # 外层捕获（因为内层重新抛出）对异常组（exception group）也是如此，只不过异常不会消亡，直到整个组都被匹配完。例如在示例 36-2 里，action2 摘走了 IndexError，action1 消费了 TypeError，而 SyntaxError 继续传播到顶层默认处理器（或者说更早的匹配 try，如果有的话）。示例 36-2. nestedexcgroup.py运行时，每个函数的 try 消费掉它匹配的那些异常，顶层处理器则为组中最后剩下的那个未匹配项打印错误信息：\$ python3 nestedexcgroup.py Got IE Got TE + Exception Group Traceback (most recent call last):...etc ...| ExceptionGroup: Nest (1 个子异常) +-+-----
--- 1 -----| SyntaxError: 3 +-----
-----无论抛出的是异常组还是单个异常，异常最终跳去的地方都取决于程序运行时的控制流。正因如此，要想到达哪里，你得知道自己去过哪里。也就是说，运行时嵌套的异常路由更多是控制流的函数，而非语句语法的函数。话虽如此，我们也可以按语法来嵌套异常处理器——一个等价的场景，下一节就来讨论。

深度理解

- 核心概念：异常在“函数调用链”上传播时，同样遵守“最近的匹配者优先”。raise1（功能正常的返回）与 raise 手动重抛是两回事：重抛 = 把已经抓住的异常再放走。
- 底层实现：抛出异常时，CPython 依次在调用栈的每一帧中查找该帧的 try-handler 表。action3 的帧没有匹配项 → 展开到 action2 的帧，其中有 except TypeError，

于是停止展开并执行那个 handler。raise (无参数) 在 handler 内部等价于"拿到当前异常并重新 raise"，它会被原样送往更大的外部 try。

·设计原因：为什么"最近的"优先而不是"声明最靠前的"优先？因为运行时嵌套反映的是"离现场越近，越了解该怎么处理"——就像球场边线外的助教比看台上的总裁更懂刚才那次失误的原因。

·实际问题：ExceptionGroup (Python 3.11+) 让"一条路径上的多种错误"可以打包集体传播，再被 except 按类型分发给不同处理器。它解决的实际问题是"asyncio/并发上下文里多个任务同时失败"的场景；但也别过度使用——能用单异常 + 类层次解决的，就不需要组。

·初学者误区：认为 except Exception as e: ... raise 与"什么都不做"等价。区别非常关键：前者在异常冒泡穿过本层之前先执行了本层的清理/日志代码；后者连这个机会都没有。另外，别以为捕获过异常后就能对异常对象为所欲为——参见下一节的局部化规则。

代码分析

```
def action3():
    print(1 + [])          # TypeErrorint list

def action2():
    try:                   # except try
        action3()
    except TypeError:
        print('Inner try') # ""
        raise              #
```

```
def action1():
    try:
        action2()
    except TypeError:
        print('Outer try') # action2

if __name__ == '__main__':
    action1()
```

```
def action3():
    raise ExceptionGroup('Nest*', [IndexError(1), TypeError(2), Sy...

def action2():
    try:
        action3()
    except* IndexError: #
        print('Got IE')

def action1():
    try:
        action2()
    except* TypeError: #
        print('Got TE')

if __name__ == '__main__':
    action1()
```

·解释器如何处理：action3() 里执行 `1 + []` 时，字节码 `BINARYOP` + 检测到不兼容类型，抛 `TypeError`。解释器向上逐帧寻找 handler：action3 的帧没有 → 进入 action2 的帧，命中 `except TypeError`，于是执行 `print('Inner try')`；随后裸 `raise` 从 `sys.exception()` 取出当前异常并原地重抛，继续向上传播到 action1 的 `except TypeError`，打印 `Outer try`。

·异常组版本的差异：raise ExceptionGroup(...) 抛出的是一个聚合对象；except IndexError 会从组里筛出所有 IndexError 子异常，把它们打包成新的子组交给 handler，而 TypeError、SyntaxError 继续留在剩下的组里传播到上一层。最终 SyntaxError: 3 无人认领，由顶层默认处理器打印那棵"组树"。

·设计思想：这两个程序演示了"同一异常能被两层看到"的唯一方式——内层主动重抛。日常开发中，内层应该只处理"自己职责范围内"的部分，把不该管的重抛出去，形成清晰的职责链。

36.4 Example: Syntactic Nesting (示例：语法嵌套)

英文原文

As discussed when we studied clause combinations of the try statement in Chapter 34, it is also possible to nest try statements syntactically by their position in your source code:

```
>>> from nestedexcnormal import action3
>>> try: ... try: ... action3() ... except TypeError: ... print('Inner try') ... raise ... except TypeError: ... print('Outer try') ...
Inner try
Outer try
```

Really, though, this code just sets up the same handler-nesting structure as, and behaves identically to, the try statements in Example 36-1. In fact, syntactic nesting works just like the cases sketched in Figures 36-1 and 36-2. The o

nly difference is that the nested handlers are physically embedded in a try block, not coded elsewhere in functions that are called from the try block. For example, nested finally handlers all fire on an exception, whether they are nested syntactically or by means of the runtime flow through physically separated parts of your code:

```
>>> try: ... try: ... action3() ... finally: ... print('Inner try') ... finally: ... print('Outer try') ...
Inner try
Outer try
Traceback (most recent call last): ... etc ...
TypeError: unsupported operand type(s) for +: 'int' and 'list'
See Figure 36-2 for a graphic illustration of this code's operation; the effect is the same, but the function logic has been inlined as nested statements here.
```

As a more comprehensive example of syntactic nesting at work, consider the file listed in Example 36-3.

Example 36-3. except-finally.py This code catches an exception if a matching one is raised and performs a finally termination-time action regardless of whether an exception occurs. This may take a few moments to digest, but the effect is the same as combining an except and a finally clause in a single try statement:

```
$ python3 except-finally.py <raise1> caught IndexError finally run ...
<noraise> finally run ... <raise2> finally run
Traceback (most recent call last): ... etc ...
SyntaxError: None
```

As we saw in Chapter 34, except and finally clauses can be mixed in the same try statement. While this, along with multiple except clauses, makes the syntactic nesting shown in this section largely academi

c, the equivalent runtime nesting is common in larger Python programs. Moreover, syntactic nesting can make the disjoint roles of `except` and `finally` explicit and might be useful for implementing alternative exception-handling behaviors.

中文翻译

正如第 34 章研究 `try` 语句子句组合时所讨论的，也可以根据 `try` 语句在源代码中的位置，进行语法上的嵌套：

```
>>> from nestedexcnormal import action3
>>> try: ... try: .. action3() ... except TypeError: ... print('Inner try') ... raise ... except TypeError: ... print('Outer try') ...
Inner try
Outer try
```

不过说真的，这段代码只是搭起了与示例 36-1 完全相同的处理器嵌套结构，行为也一模一样。事实上，语法嵌套的运行方式与图 36-1 和图 36-2 勾勒的情形完全相同。唯一的差别在于：嵌套的处理器是字面上嵌在某个 `try` 块内部，而不是写在从该 `try` 块调用的、物理上分离的函数里。例如，无论在语法上嵌套，还是通过代码中物理分开部分的运行时流嵌套，嵌套的 `finally` 处理器在一个异常发生时都会全部触发：

```
>>> try: ... try: ... action3() ... finally: ... print('Inner try') ... finally: ... print('Outer try') ...
Inner try
Outer try
Traceback (most recent call last):...etc
...TypeError: unsupported operand type(s) for +:'int' and 'list'
```

参见图 36-2 以直观理解这段代码的运行；效果相同，只是函数逻辑在这里被内联（`inlined`）成了嵌套语句。作为一个更全面的语法嵌套实例，请看示例 36-3 列出的文件。示例 36-3. `except-finally.py` 这段代码在抛出匹配的异常时捕获它，并且无论是否发生异常都执行 `finally` 的终止

时动作。这一点可能需要花点时间消化，但效果与把 `except` 和 `finally` 子句组合在单个 `try` 语句里是完全一样的：`$ python3 except-finally.py <raise1> caught IndexError finally run ... <noraise> finally run ... <raise2> finally run` `Traceback (most recent call last):...etc...SyntaxError: None` 正如第 34 章所见，`except` 和 `finally` 子句可以混在同一个 `try` 语句里。尽管这（再加上多个 `except` 子句）让本节展示的语法嵌套在很大程度上变得“学术化”，但等价的运行时嵌套在大型 Python 程序中相当常见。此外，语法嵌套可以显式区分 `except` 与 `finally` 的不同角色，还可能对实现“替代性异常处理行为”有用。

深度理解

- 核心概念：语法嵌套与运行时嵌套只是“同一座冰山的两面”。语法嵌套是“在代码里套娃”，运行时嵌套是“通过函数调用关系套娃”——两者的异常路由规则完全一致。
- 底层实现：`finally` 在字节码层面相当于“每离开一次 `try` 作用域触发一次的清理钩子”。三层 `finally` 嵌套时，异常传播路径上每一层都会在自己的清理代码执行完毕后继续上抛，顺序是“内层先跑，外层后跑”（就像剥洋葱，从里往外）。而 `except` 一旦命中，栈展开立即停止——这是 `except` 与 `finally` 在实现上最本质的分界。
- 设计原因：Python 允许 `try/except/finally` 混写在同一个语句里（自 Python 2.5），使得很多场景不需要语法嵌套。保留“嵌套写法”的意义在于：明确表达“这两段收尾拥有不同的执行时机语义”；它也便于在包一层时插入不同的异常处理策略。

·实际问题：在大型程序里，你几乎总是遇到"运行时嵌套"（框架的 try 包裹着你的代码），所以理解"异常会冒泡穿过多少层 finally、最终停在哪层 except"对排查线上问题是基本能力。

·初学者误区：把 raise 写成入门练习参考示例 36-3——注意当 SyntaxError 被 raise2 抛出后，finally run 照样打印，这证明 finally 不拦截异常。另一个常见误解是"有 try 就一定能捕获"，实际上如果你在 try 外面 raise，异常照样逃逸。

代码分析

```
def raise1(): raise IndexError
def noraise(): return
def raise2(): raise SyntaxError

for func in (raise1, noraise, raise2):
    print(f'<{func.__name__}>')
    try:
        func()
    except IndexError:
        print('caught IndexError')
    finally:
        print('finally run')
    print('...')
```

·解释器如何处理：循环三轮，每轮把函数放进"内层 try（负责捕获 IndexError）→ 外层 finally（保证清理）"的守卫里。

·第一轮 raise1()：抛 IndexError → 内层 except IndexError 命中，打印 caught IndexError，异常被消费；随后

走 finally, 打印 finally run; 接着正常打印 ...。

·第二轮 `noraise()`: 无异常, 跳过 `except`, 直接执行 `finally` (打印 `finally run`) , 再打 ...。

·第三轮 `raise2()`: 抛 `SyntaxError`, 内层 `except IndexError` 不匹配 → 不消费; `finally` 照常运行 (打印 `finally run`) 但拦不住异常 → 异常继续上抛到顶层, 打印 `Traceback ... SyntaxError: None`, 之后的 ... 不会执行。

·设计思想: 一个小循环同时演示了 `except` 与 `finally` 的三种组合情形 (捕获、无异常、未捕获但清理)。这正是"一个 `try` 里混用子句"所隐含的语义——`finally` 是"不管怎样都跑", `except` 是"只有匹配才跑"。

36.5 Exception Idioms (异常惯用法)

英文原文

We've seen the mechanics behind exceptions. Now, let's survey the ways they are typically used. Some of these are reviews of roles we've explored in earlier chapters, collected here as part of a referable set.

中文翻译

我们已经了解了异常背后的机制。现在, 让我们一起总览一下它们通常的用法。其中有些是对前面章节探索过的角色的回顾, 这里把它们收集起来, 作为一套可随时查阅的参考。

深度理解

·核心概念：本节是"异常的模式语言"——把异常少见的几种用途（当"go to"、当信号、当返回值替身、当清理钩子、当调试兜网）集中陈列。

·底层视角：这些惯用法之所以能用，都建立在异常的两条机械性事实上：(1) 异常是跨函数边界的控制转移；(2) 异常携带对象，可以承载信息。

·设计原因：作者把这些惯用法收在核心部分的结尾，是为了让你在进入"应用开发"前形成选择异常还是选择返回值的判断力。

·初学者误区：以为异常只能表示"出错"。下一节直接纠正这一点——"并非所有异常都是错误"。

36.6 Idiom: Breaking Out of Multiple Nested Loops — "go to"（惯用法：跳出多重嵌套循环——"go to"）

英文原文

As mentioned at the start of this part of the book, exceptions can often be used to serve the same roles as other languages' "go-to" statements to implement more arbitrary control transfers. Exceptions, however, provide a more structured option that localizes the jump to a specific block of nested code. In this role, raise is like "go to," and except clauses and exception n

ames take the place of program labels. You can only jump out of code wrapped in a try this way, but that's a crucial feature—truly arbitrary "go to" statements can make code extraordinarily difficult to understand and maintain ("spaghetti code" in developer lingo). For example, Python's break statement exits just the single closest enclosing loop, but we can always use exceptions to break out of more than one loop level if needed, as in Example 36-4.

Example 36-4. breaker.py

When run, the raise in the for breaks out of three nested loops immediately: `$ python3 breaker.py loop3: 0 loop3: 1 loop3: 2 loop3: 3 continuing i=4` If you change the raise in this to break, you'll get an infinite loop because you'll break only out of the most deeply nested for loop, and wind up in the second-level while loop nesting. The code would then print "loop2" and start the for again. Make the mod to see for yourself—but get ready to type Ctrl+C to stop the code! Also, notice that variable i is still what it was in for after the try statement exits. As previously noted, variable assignments made in a try are not undone in general, though as we've seen, exception instance variables listed in except clause as headers are localized to that clause, and the local variables of any functions that are exited as a result of a raise are discarded. Technically, active functions' local variables are popped off the call stack, and the objects they reference may be garbage-collected as a result, but this is an automatic st

中文翻译

如本书这一部分开头所述，异常常可扮演其他语言“goto”（跳转）语句所扮演的角色，实现更任意的控制转移。不过异常提供了一种更有结构的方案：把跳转限定到某一块嵌套代码内部。在这种角色里，raise 就像“go to”，而 except 子句与异常名扮演了程序标签（label）的角色。你只能以这种方式跳出被 try 包裹的代码——但这是一个至关重要的特性：真正随心所欲的“go to”会让代码极难理解和维护（开发者行话里的“意大利面条代码”spaghetti code）。例如，Python 的 break 语句只退出最近的单一包绕循环，但如果需要，我们总能用异常跳出多层循环，如示例 36-4 所示。示例 36-4. breaker.py 运行时，for 里的 raise 会立即跳出三层嵌套循环：\$ python3 breaker.py loop3: 0 loop3: 1 loop3: 2 loop3: 3 continuing i=4 如果把这里的 raise 改成 break，你会得到死循环——因为 break 只跳出嵌套最深的 for 循环，然后进入第二层的 while 循环嵌套中。代码会打印“loop2”并重新开始 for。不信可以自己改改试试——但准备好按 Ctrl+C 来终止代码！另外注意，try 语句退出后，变量 i 仍然保持着它在 for 里的值。如前所述，一般而言在 try 中所做的变量赋值不会被撤销；不过正如我们所见，写在 except 子句头部作为“接管变量”的异常实例是局部于该子句的，而因 raise 而退出的任何函数的局部变量则被丢弃。从技术上讲，活跃函数的局部变量会从调用栈中弹出，它们引用的对象因此可能被垃圾回收——但这是自动进行的步骤。

深度理解

·核心概念：用异常实现"多级跳出"。break 只管一层循环，要想一次挣脱多层，最干净的途径就是 raise 一个专用异常，在外层一个 except 里收口。类比：电梯紧急按钮——平时一层层下太慢，按一次直接送到地面层。

·底层实现：raise 触发的是异常展开（unwinding）机制——内层 for / while 的帧和循环指令直接跳过，程序指针跳到外层 except 处。正因为走的是"异常展开"而非"循循环跳出"，所有循环的控制位都不需要逐一复位。

·设计原因：为什么默认不提供 goto？因为 goto 可以跳去任何地方（包括往前跳），会让"程序接下来有什么状态"变得不可预测。异常版 goto 只允许向外跳，且跳之前一定经过 finally——这是 Python 对"结构化控制转移"的取舍。

·实际问题：这个惯用法在"搜索/遍历并提前终止"（如找到目标就退出多层循环）的代码里非常实用；但它也是危险牌——滥用会让流程晦涩。现代 Python 里，优先考虑把内部逻辑提取成函数 + return，把 raise 留给真正跨层的情况。

·初学者误区：误以为 raise 之后循环变量会被"回滚"。事实是：try 抛出异常后，正值传递给后续代码——示例中 i = 4 证明了逻辑无罪。另一个误区是以为"异常离开作用域时局部对象一定被回收"：实际上只有函数级的局部变量弹栈时，若无其他引用才会被 GC；模块级和 except as 里

的变量都不在此列。

·补充：except 子句头的异常实例（except TypeError as e:）在 Python 3 里是"try 块内临时绑定、子句结束时删除"的，所以出了 except 块再访问 e 会得到 NameError（或引用已遗弃对象），这是初学者常踩的坑。

代码分析

```
class Exitloop(Exception): pass

try:
    while True:
        while True:
            for i in range(10):
                if i > 3: raise Exitloop    # break
                print('loop3: %s' % i)     # raise
            print('loop2')
        print('loop1')
except Exitloop:
    print('continuing')    # pass
print(f'{i=}')            #
```

·解释器如何处理：前 4 次迭代打印 loop3: 0..3；第 5 次迭代 i=4 时命中 if i > 3，执行 raise Exitloop。异常展开机制从最内层 for 的帧直接一路弹出整个三层循环结构，直到外层 except Exitloop 匹配；中间的 print('loop2')、print('loop1') 全都跳过了（这就是效果上是"多级 break"的原因）。随后打印 continuing 与 i=4——因为变量 i 只是普通局部名，在 range 中被重新绑定过，异常不撤销赋值。

·设计思想：为"退出信号"定义一个专用异常类（class Exitloop(Exception)），比复用内置异常更安全、更语义化

——你不会误捕获到别的代码抛出的 `KeyError` 之类。这是一种“自定义控制流信号”的惯用法。

36.7 Idiom: Exceptions Aren't Always Errors (惯用法：异常并非总是错误)

英文原文

In Python, all errors are exceptions, but not all exceptions are errors. For instance, we saw in Chapter 9 that file object read methods return an empty string at the end of a file. In contrast, the built-in input function—which we first met in Chapter 3 and deployed in an interactive loop in Chapter 10—reads a line of text from the standard input stream, `sys.stdin`, on each call and raises the built-in `EOFError` at end-of-file. Unlike file methods, this function does not return an empty string—an empty string from input means an empty line. Despite its name, though, the `EOFError` exception is just a signal in this context, not an error. Because of this behavior, unless the end-of-file should terminate a script, input often appears wrapped in a try handler and nested in a loop, as in the following code: `while True: try: line = input() # Read line from stdin except EOFError: break # Exit loop at end-of-file else:...process next line here...` Several other built-in exceptions are similarly signals, not errors—for example, calling `sys.exit()` and pressing `Ctrl+C` on your keyboard raise

SystemExit and KeyboardInterrupt, respectively. Python also has a set of built-in exceptions that represent warnings rather than errors; some of these are used to signal the use of deprecated (soon to be phased out) language features. See the standard-library manual's description of built-in exceptions for more information, and consult the warnings module's documentation for more on exceptions raised as warnings.

中文翻译

在 Python 中，所有错误都是异常，但并非所有异常都是错误。例如，我们在第 9 章看到，文件对象的 read 方法在文件结尾返回空字符串。相比之下，内置函数 input——我们在第 3 章首次结识、并在第 10 章的交互式循环中用到——每次调用从标准输入流 sys.stdin 读取一行文本，并在文件末尾 (end-of-file) 抛出内置的 EOFError。与文件方法不同，这个函数不会返回空字符串——input 返回空字符串意味着“空行”。因此，尽管名为 EOFError，这个异常在上下文中只是一个信号 (signal)，而非错误。正因如此，除非“文件结束”应该终止脚本，否则 input 常常被包在 try 处理器里、嵌套在一个循环中，正如下面的代码：

```
while True:
    try: line = input() # 从标准输入读取一行
    except EOFError: break # 在文件末尾跳出循环
else: ...在这里处理下一行
```

...其他几个内置异常同样是“信号”而非“错误”——例如调用 sys.exit() 会抛 SystemExit，在键盘上按 Ctrl+C 会抛 KeyboardInterrupt。Python 还有一组表示的警告 (warnings) 而非错误的内置异常；其中一些被用来提示使用了已弃用 (即将退役) 的语言特性。更多信息见标准库手册对内置

异常的说明；被作为警告抛出的异常的更多细节，请查阅 `warnings` 模块的文档。

深度理解

·核心概念：异常是 Python 的通用通知机制——不只报错，还用于表示"事件"。EOFError、SystemExit、KeyboardInterrupt 都是"控制流信号"。

·底层实现：`sys.exit()` 的实现本质就是 `raise SystemExit(code)`；解释器对 SystemExit 特殊处理——捕捉该异常、把它的参数作为进程退出码。KeyboardInterrupt 由信号处理器（SIGINT）转成 Python 异常。`input()` 在读不到数据时主动 `raise EOFError`，而不是返回哨兵值。

·设计原因：为什么 `input` 不学文件对象返回空串表示"结束"？因为 `input` 的语义里空串是合法输入（空行）。既然返回值必须"永远合法"，文档里就只剩"用异常送出结束信号"这一条路——这正是 36.8 节"函数用 `raise` 传状态"的原理源头。

·实际问题：写 CLI 工具、数据管道时，"循环读行直到 EOF"是和 `Break` 一样常见的模式；若忘记 `except EOFError`，脚本在输入流结束时就会打出难看的 `traceback`。

·初学者误区：看见以 `Error` 结尾的类名就以为"程序错了"。EOFError 在这里根本不是错误——你要判断的是"它被人捕获了吗"。另一个误区：用空 `except` 一并吞掉 `KeyboardInterrupt` (`Ctrl+C`)，会让用户无法中断死循环的程序，行为既可怕又难调试。

36.8 Idiom: Functions Can Signal Conditions with raise (惯用法: 函数可以用 raise 传达状态)

英文原文

User-defined exceptions can also signal nonerror conditions. For instance, a search routine can be coded to raise an exception when a match is found instead of returning a status flag for the caller to interpret. In the following abstract code, the try/except/else exception handler does the work of an if/else return-value tester:

```
class Found(Exception): pass
def searcher():
    if ...success...:
        raise Found() # Raise exceptions instead of returning flags
    else:
        return
```

```
try:
    searcher()
```

```
except Found: # Exception if item was found...success
    ...
```

```
else: # else returned: not found...failure...
```

More generally, such a coding structure may also be useful for any function that cannot return a sentinel value to designate success or failure. In a widely applicable function, for instance, if all objects are potentially valid return values, it's impossible for any return value to signal a failure condition. Exceptions provide a

way to signal results without a return value:

```
class Failure(Exception): pass
def searcher():
    if ...success...:
        return ...founditem...
    else:
        raise Failure()
try:
    item = searcher()
except Failure:
    ...not found...

else:
    ...use item here...
```

Because Python is dynamically typed and polymorphic to the core, exceptions, rather than sentinel return values, are the generally preferred way to signal such conditions.

中文翻译

用户自定义异常也可以传达非错误状态。例如，一个搜索例程可以被编写成“找到匹配时就抛一个异常”，而不是返回一个让调用方去解读的状态标志。在下面这段抽象代码中，try/except/else 异常处理器承担了 if/else 返回值测试器的工作：

```
class Found(Exception): pass
def searcher():
    if ...成功...:
        raise Found() # 用抛异常代替返回标志
    else:
        return ...失败处理...
try:
    searcher()
except Found:
    # 抛了异常 = 找到了
    ...成功处理...
else:
    # else 执行了 = 没找到
    ...失败处理...
```

更一般地，这种编码结构对任何“无法用哨兵值（sentinel value）表示成功或失败”的函数都是有用的。例如在一个适用范围极广的函数里，如果任何对象都可能是合法返回值，那么就不可能用某个返回值来表示失败条件。异常提供了一种“不需要返回值就能传达结果”的方式：

```
class Failure(Exception): pass
def searcher():
    if ...成功...:
        return ...找到的项目...
    else:
        raise Failure()
try:
    item = searcher()
except Failure:
    ...未找到处理...
else:
    ...在这里使用 item...
因为 P
```

ython 是动态类型、且多态到骨子里的语言，传达此类状态时，异常（而非哨兵返回值）通常是更受青睐的方式。

深度理解

·核心概念：当"返回值"被所有合法值占满、没有空闲的哨兵位时，异常是表示"特殊情况"的唯一可靠通道。也可以反过来：在找不到时抛 Found——让"成功"成为异常，打破常规以强调"这是一次罕见事件"。

·底层实现：这两种写法都本质上是"用异常展开替代显式分支"——异常抛出时栈自动回卷，else 子句只在 try 体正常结束（没有抛、没有 break/continue/return）时执行。raise 与 return 相比，承担了"跳走 + 携带状态 + 触发清理"三合一职责。

·设计原因：动态语言里函数可以返回任意对象，因此"返回值约定"很难文档化为铁的纪律；而异常属于异常契约，调用方如果不处理，程序就会如实报错（fail loudly），不像哨兵值那样容易被默默忽略。

·实际问题：这正是许多库的通用错误处理设计（例如找不到键时 py:KeyError、找 URL 时 HTTPError）。不过要克制：日常"查找不到"用 None/False 即可，只有"失败必须显式处理、且无法用返回值表示"时才动用异常。

·初学者误区：以为"抛异常"就一定比"返回 None"高级。事实是：异常适合"控制流转移"（跨多层很重要），返回 None 适合"紧邻的、常规的缺省值"。另外注意 else 子句的执行时机——它要求 try 体成功执行完，混入 return 的 raise 写法时最容易把 else 误判为"总是执行"。

36.9 Idiom: Closing Files and Server Connections (惯用法: 关闭文件与服务器连接)

英文原文

We encountered examples in this category in Chapter 34. As a review, exception processing tools are also commonly used to ensure that system resources are finalized, regardless of whether an error occurs during processing or not. For example, some servers require connections to be closed in order to terminate a session. Similarly, output files may require close calls to flush their buffers to disk for waiting consumers; input files may consume file descriptors if not closed; and CPython closes open files when garbage-collecting them, but this isn't always predictable or reliable. As we saw in Chapter 34, the most general and explicit way to guarantee termination actions for a specific block of code is the try/finally combination:

```
myfile = open('somefile','w') try:...process myfile... finally: myfile.close()
```

As we also saw, some objects make this potentially easier by providing context managers that terminate or close resources for us automatically when run by the with statement:

```
with open('somefile','w') as myfile:...process myfile...
```

If you want to know which option is better, flip back to "The Termination-Handlers Shoot-Out"—and draw your own conclusions.

中文翻译

我们在第 34 章遇到过这一类别的示例。作为回顾，异常处理工具也常被用来确保系统资源被收尾——无论处理过程中是否出错。例如，有些服务器要求关闭连接以结束会话。类似地，输出文件可能需要 `close` 调用来把缓冲区刷新到磁盘供等待的消费者读取；输入文件如果不关闭可能持续占用文件描述符（file descriptor）；CPython 在垃圾回收打开的文件时会关闭它们，但这并不总是可预测或可靠的。正如我们在第 34 章看到的，保证某块代码终止动作的最通用、最显式的方式是 `try/finally` 组合：`myfile = open('somefile','w')` `try:...处理 myfile...finally: myfile.close()` 我们同样看到，有些对象通过提供上下文管理器（context manager）让这件事更省事——由 `with` 语句运行它们时，会自动替我们终止或关闭资源：`with open('somefile','w') as myfile:...处理 myfile...` 想知道哪种方案更好？翻回第 34 章 "终止处理器的终极对决"（The Termination-Handlers Shoot-Out），然后自己做结论。

深度理解

·核心概念：资源清理（文件、连接、锁、临时目录.....）必须"无论成败都执行"，这是 `finally` / `with` 存在的意义。
·底层机制：`with` 语句在字节码层编译为"进入时调用 `enter`，离开时（包括异常路径）调用 `exit`"，`exit` 内部就是 `finally` 语义的再包装。

· with 与 try/finally 的取舍（本章重点，务必吃透）：

对比维度	try/finally	with（上下文管理器
简洁度	样板代码多，要自己 close()	一行搞定，exit 自动清理
灵活性	可在 finally 里做任意清理逻辑（发邮件、记日志、回滚）	只能做对象定义好的 exit
异常上下文	能同时用 except 捕获同一块代码的异常	捕获异常需再包一层，或自定义 contextmanager
适用对象	任何代码块	实现了上下文管理器协议的对象
可复用性	每次手写	一写永用（可进库）

经验法则：对象自带上下文管理器就用 with；否则用 try/finally。真实世界中约九成资源清理场景，with 是"正确且省心"的选择。

·设计原因：为什么 CPython 的 GC 自动关文件"不可靠"？因为 CPython 用引用计数——对象在不可预测的时刻、以不可预测的顺序被回收；靠 GC 收尾既不及时也不确定。而 with / finally 提供了确定性清理，这才是资源管理的正道。

·实际问题：服务端连接泄漏、日志文件没 flush、临时文件没删——这些线上事故大多源于程序在异常路径上没有关闭资源。用 with 后这些 bug 的空间被压到最小。

·初学者误区：以为"反正程序退出时系统会回收，不用关闭"。文件描述符在长运行进程里是用一个少一个；更重要的是 flush 时机——未 close 的输出缓冲可能丢失，消费方永远等不到数据。也常见误区：认为 with 不能处理异常——实际上 exit 收到异常信息，你可以在里面吃掉它

或传播它。

·补充（现代实践）：自 Python 3.4 `contextlib.suppress`、3.8 起的 `contextlib.AbstractContextManager`、以及 `@contextmanager` 装饰器，都让“定制清理/恢复”更优雅；`with` 还支持在同一行开多个资源：`with open(a) as fa, open(b) as fb:`——这比嵌套写更清晰。

36.10 Idiom: Debugging with Outer try Statements（惯用法：用外层 try 语句调试）

英文原文

You can also make use of exception handlers to replace Python's default top-level exception-handling behavior. By wrapping an entire program (or a call to it) in an outer try in your top-level code, you can catch any exception that may occur while your program runs, thereby subverting the default program termination. In the following, the empty except clause catches any uncaught exception raised while the program runs. To get hold of the actual exception that occurred in this mode, fetch the `excinfo` function call result from the built-in `sys` module; it returns a tuple whose first two items contain the currently handled exception's class and the instance object raised (more on `sys.excinfo` in a moment):

```
try:...run program...except: # All uncaught exceptions come here
import sys
print('uncaught', sys.exc_info())
```

tl', sys.excinfo()[0], sys.excinfo()[1]) This structure is commonly used during development to keep programs active even after errors occur. It's also used when testing other program code, as described in the next section: coded within a loop, this structure allows you to run additional tests without having to restart.

中文翻译

你也可以利用异常处理器来替换 Python 的默认顶层异常处理行为。在顶层代码里，把整个程序（或对它的调用）包在外层 try 中，就能捕获程序运行期间可能抛出的任何异常，从而绕开默认的“程序终止”行为。在下面这段代码里，空的 except 子句捕获程序运行期间抛出的任何未捕获异常。要在这个模式下拿到实际发生的异常，请从内置 sys 模块取回 excinfo 函数的调用结果；它返回一个元组，前两项分别是被处理异常的类与被抛出的实例对象（关于 sys.excinfo 稍后细讲）：`try:...运行程序...except: # 所有未捕获的异常都到这里``import sys print('uncaught!', sys.excinfo()[0], sys.excinfo()[1])` 这种结构在开发阶段很常用，用来让程序在出错后依然保持运行。它也用于测试其他程序代码（下一节会讲到）：放在循环里时，这种结构允许你继续运行后续测试而无需重启。

深度理解

·核心概念：把“程序级默认处理器”替换成“程序内的兜底句柄”——让异常不杀死进程，而是被记录/继续。这是开发期护航技术，不是生产期推荐做法。

- 底层实现：空的 `except:` 捕获 `BaseException` 之下的一切（包括 `KeyboardInterrupt/SystemExit`）。`sys.excinfo()` 返回 `(type, value, traceback)` 三元组，其中 `traceback` 是回溯对象。自 Python 3.11 起也可用 `sys.exception()` 只取异常实例。
- 设计原因：开发期让程序"带病继续跑"，能一次验证多个后续代码路径是否还能用——省去反复重启的手工劳动。
- 实际问题：此模式的风险与它带来的便利成正比——吞掉异常之后，错误被隐藏，极易把 bug 带到发布。更现代的做法是：`except` 里把异常写日志（`logging.exception()`），而不是只 `print`。
- 初学者误区：把这种"兜底 `catch`"直接搬进生产脚本。生产环境通常应该让异常终止程序（`fail fast`），让监控报警而不是默默吞掉。尤其是不要包住所有代码还什么都不做——那等于给程序装了个"空气刹车"。

36.11 Idiom: Running In-Process Tests (惯用法：进程内运行测试)

英文原文

Some of the coding patterns we've just seen can be combined in a test-driver script that tests other code imported and run within the same process (i.e., program run). The following partial and abstract code sketches the general model: `import sys; log = open('testlo`

g','a') from testapi import moreTests, runNextTest, testName def testdriver(): while moreTests(): try: runNextTest() except: print('FAILED', testName(), sys.excinfo()[2], file=log) else: print('PASSED', testName(), file=log) testdriver()

The testdriver function here cycles through a series of test calls. Because an uncaught exception in any of them would normally kill this test driver, tests are wrapped in a try to continue the testing process if a test fails. The empty except catches any uncaught exception generated by a test case and uses sys.excinfo to log the exception to a file. The else clause is run when no exception occurs—the test succeeds case. Such boilerplate code is typical of systems that test imported functions, modules, and classes. In practice, though, testing can be much more sophisticated. For instance, to test external programs, you could instead check status codes or outputs generated by program-launching tools such as os.system and os.popen, which were used earlier in this book and are covered in Python's standard-library manual. Such tools do not generally raise exceptions for errors in the external programs—in fact, the test cases may run in parallel with the test driver. At the end of this chapter, we'll also briefly explore more complete testing frameworks provided by Python, such as doctest and PyUnit, which provide tools for comparing expected outputs with actual results.

中文翻译

我们刚看过的某些编码模式可以组合成一个测试驱动（test-driver）脚本：它在同一个进程（即同一个程序运行）里导入并运行其他代码。下面这段不完整、抽象的代码勾勒出通用模型：

```
import sys
log = open('testlog','a')
from testapi import moreTests, runNextTest, testName
def testdriver():
    while moreTests():
        try:
            runNextTest()
        except:
            print('FAILED', testName(), sys.excinfo()[2], file=log)
        else:
            print('PASSED', testName(), file=log)
    testdriver()
```

这里的 testdriver 函数循环执行一系列测试调用。由于任何一个测试里的未捕获异常通常都会杀死这个测试驱动器，测试被包进 try，以便某个测试失败时测试过程还能继续。空的 except 捕获测试用例产生的任何未捕获异常，并用 sys.excinfo 把异常记录到文件；else 子句在没有异常发生时执行——也就是测试成功的分支。这类样板代码是“测试被导入的函数、模块、类”这类系统的典型写法。但实际上，测试可以复杂得多。例如，要测试外部程序，可以改而检查由“程序启动工具”（如 os.system、os.popen，本书前面用过，标准库手册有介绍）产生的状态码或输出。这类工具通常不会针对外部程序中的错误抛异常——事实上，测试用例可能与测试驱动器并行运行。到本章末尾，我们还会简要探讨 Python 提供的更完整的测试框架，如 doctest 和 PyUnit，它们提供了把期望输出与实际结果做比较的工具。

深度理解

·核心概念：测试驱动的 Notion——一个测试用例失败，

不能终止整个测试套件。为达成"一个失败不影响其余"，把每个用例放进 `try`，用空 `except` 兜底记录，用 `else` 标记成功。

·底层实现：`sys.excinfo()[2]` 是 `(type, value)`——异常类与实例；打印它们即可回溯定位。`file=log` 把内容写入已打开的日志文件而不是 `stdout`。注意 `log` 在模块顶层用普通 `open` 打开只追加（'a'），意味着驱动与用例共享同一进程——这就是"进程内测试"。

·设计原因：为何"捕获一切进行跑批"？因为单元测试的本意是收集失败而不是第一个失败就停。`doctest` 与 `PyUnit` 最终取代这种手写样板的原因也在此——它们把"断言、报告、收集、统计"标准化了。

·实际问题：这个模式至今仍以各种形式存在于 CI 系统和自定义测试运行器里。它让我们看到"空 `except` + `excinfo`"短暂但不失体面的用武之地：静默写日志 + 继续跑是收集型工具的合理设计。

·初学者误区：试图用 `except`：在测试里"喝了"失败——把记录 `exception` 改成记录"发生了什么"之外，别忘记记录栈（`traceback.formatexc()`）。另外注意"空 `except` 也会吞 `KeyboardInterrupt`"，如果测试期间用户想 `Ctrl+C` 中断，得先放行它。

36.12 More on `sys.excinfo` (深入 `sys.excinfo`)

英文原文

The `sys.excinfo` result used in the last two sections allows an exception handler to generically gain access to the exception being handled. This is especially useful when using the empty `except` clause to catch everything blindly because it allows you to determine what was raised: `try:...except: # sys.excinfo()[0:2]` are the exception class and instance. If no exception is being handled, this call returns a tuple containing three `None` values. Otherwise, the values returned are (type, value, traceback), where:

- type is the class of the exception being handled.
- value is the class instance that was raised.
- traceback is a traceback object that represents the call stack at the point where the exception originally occurred, and may be used by the `traceback` module to generate error messages.

As we saw in Chapter 35, `sys.excinfo` can also sometimes be useful to determine the specific exception type when catching exception category superclasses. As we've also learned, though, because in this case you can also get the exception type by fetching the class attribute or type of the instance obtained with the `as` clause, `sys.excinfo` is rarely useful outside the empty `except: try:...except General as instance: # instance.class or type(instance)` is the exception class # but `instance.method()` does the right thing for this instance. As we've seen, using `Exception` for the General exception name

here would catch all nonexit exceptions; it's similar to an empty except but less extreme and still gives access to the exception instance and its class. Even so, leveraging polymorphism by calling the instance's methods is often a better approach than testing exception types.

中文翻译

前两节用到的 `sys.excinfo` 结果允许异常处理器以通用方式访问当前被处理的异常。当使用空的 `except` 子句"盲目地"捕获一切时，这一点尤其有用，因为它让你能确定到底抛出了什么：`try:...except: # sys.excinfo()[0:2]` 分别是异常的类和实例如果没有异常正在被处理，这个调用返回一个包含三个 `None` 值的元组。否则，返回的值是 `(type, value, traceback)`，其中：

- `type`：被处理异常的类。
- `value`：被抛出的类实例。
- `traceback`：表示异常最初发生点调用栈的回溯对象 (`traceback object`)，可被 `traceback` 模块用于生成错误信息。如第 35 章所见，捕获异常类别超类 (`superclass`) 时，`sys.excinfo` 有时也能用来确定具体的异常类型。不过我们也学到，在这种情况下，其实可以通过读取 `as` 子句拿到的实例的 `class` 属性，或对该实例调用 `type`，同样得到异常类型；因此，除了空的 `except`，`sys.excinfo` 很少有用武之地：`try:...except General as instance: # instance.class` 或 `type(instance)` 就是异常的类# 但 `instance.method()` 会对这个实例做正确的事正如我们所看到的，这里用 `Exception` 作为 `General` 异常名会捕获所有"非退出类"异常；它与空 `except` 相似，但没那么极端，而且仍能拿到异常实例及其类。即便如此，利用多态——调用

实例的方法——通常比测试异常类型更好。

深度理解

·核心概念：`sys.excinfo()` 是在"捕获一切但不知道是什么"时，告诉你"到底是什么"的关键 API。三元组 (type, value, traceback) 的信息密度高于 `except ... as e` (后者拿不到 traceback 对象)。

·底层实现：`sys.excinfo` 返回当前线程正在处理的异常。每个线程都有一个"当前异常"槽位；traceback 是链式对象 (tbnext 链)，由 raise 时逐帧记录调用栈生成。注意：在 except 块结束后，`sys.excinfo()` 的一部分会复位，因此若要留档，应先把它复制出来。

·设计原因：为什么提供 as 子句还要留 `excinfo`？因为 as 只绑定实例；回溯对象不暴露给 as。`excinfo` 的第三项才是 traceback 模块加工出"文件名:行号"原始素材的地方。而 traceback 对象正是回溯对象 (traceback object) 这一话题的核心。

·实际问题：日志系统 (`logging.exception`)、调试器、错误收集器都要靠它。它的价值集中在"空 except 捕获 + 后续处理"这种宽网场景。

·初学者误区：在 `except General as instance:` 里还要去 `sys.excinfo()[1]` 取实例——这纯属重复劳动；用 `instance` 就够了。更优的思维是面向多态：调用实例的方法（如区分不同类型异常时写个子类化方法），而不是 `type(instance) is ...` 的分支判断。

补：回溯对象与异常链（本书对应主题，属 Python 3 标准行为补充） - 每个异常实例都有 `.traceback` 属性（回溯对象链，反映原始抛出点）。 - 当你在 `except` 里处理一个异常、又抛出另一个时，新异常默认在 `.context` 里记下旧异常，显示为 "During handling of the above exception, another exception occurred"（隐式链）。 - 若用 `raise B() from A` 显式声明，则旧异常记在 `.cause`，显示为 "The above exception was the direct cause of the following exception"（显式链，`printexc`/日志都可看到）。 - 用 `raise B() from None` 可以切断链，隐藏不想让用户看到的内部异常。 - 文件里正文虽未展开，但理解这三者（`traceback` / `context` / `cause`）是"异常设计"的必修课，将在技术拓展中再述。

36.13 The `sys.exception` Alternative—and `diss` (`sys.exception` 替代方案——以及吐槽)

英文原文

As yet another option, a new call added in Python 3.11, `sys.exception`, returns just the exception instance raised—the same object assigned to the variable listed after `as` in `except` clauses, and equivalent to the second item in the `sys.excinfo` result (i.e., `sys.excinfo()[1]`). Hence, the following work the same in a `try: except ...`: `print('uncaught!', sys.excinfo()[0], sys.excinfo()[1])` `except ...: print('uncaught!', type(sys.exception()), sys.exception())` More generally, there are now three way

s to obtain the same information about a caught exception in try (two of which in the following are nested in a tuple to match excinfo and display with repr instead of str):

```
>>> class E(Exception): pass ... >>> try:
...     raise E('info') ... except E as X: ... print((type(X), X)) ..
. print(sys.excinfo()[2]) ... print((type(sys.exception()),
sys.exception())) ... (<class'main.E'>, E('info')) (<class'
main.E'>, E('info')) (<class'main.E'>, E('info'))
```

When using `sys.exception`, the exception class is available from the instance via `class` or `type` (as shown), and the traceback is normally present in the exception instance's `traceback` object. Though largely trivial, the new call avoids an index or slice when only the instance is needed. Less pleasantly, with the addition of `sys.exception`, the `sys.excinfo` call has also been branded "old-style" in Python's docs, but doing this for the sake of a redundant call added just over a year ago seems solely by a preference for newness—if not pinnacle. In any case, you're welcome and encouraged to use either call in your code, but this book generally recommends tools that are standard, customary, and widely established on the web.

中文翻译

(接续自区段标题之后的完整译文，与上文语块逐段对应：)

作为又一个选项，Python 3.11 新加的一个调用 `sys.exception` 返回恰好就是那被抛出的异常实例——与 `except` 子

句中 `as` 之后变量被赋值的同一个对象，也等价于 `sys.excinfo` 结果的第二项（即 `sys.excinfo()[1]`）。因此，下面两种写法在一个 `try` 中所做的工作完全相同：`except ...: print('uncaught!', sys.excinfo()[0], sys.excinfo()[1])` `except ...: print('uncaught!', type(sys.exception()), sys.exception())` 更一般地说，现在获取一个被捕获异常相同信息已有三种方式——下面举例时，前两条信息被放进元组里以对齐 `excinfo` 的格式，并用 `repr` 而不是 `str` 来显示：

```
>>> class E(Exception): pass ... >>> try: ... raise E('info') ... except E as X: ... print((type(X), X)) ... print(sys.excinfo()[2]) ... print((type(sys.exception()), sys.exception())) ... (<class'main.E'>, E('info')) (<class'main.E'>, E('info')) (<class'main.E'>, E('info'))
```

当使用 `sys.exception` 时，异常类可以从实例身上通过 `class` 或 `type` 拿到（如上所示），回溯信息通常也在异常实例的 `traceback` 对象里。仅就这个新调用本身而言意义不大，但当你只需要实例、而不想再多写一个索引或切片（`sys.excinfo()[1]`）时，它确实省了一点事。不那么令人愉快的是：随着 `sys.exception` 的加入，`sys.excinfo` 这个调用也已经被 Python 官方文档贴上了“旧式（old-style）”的标签。为一个一年多前才加入的、功能重复的调用，就如此操刀，看上去既带着成见又在制造分裂——说成“软件年龄歧视”（software ageism）也不为过。当然，你的代码里用调用哪个都用随你，欢迎并鼓励；但这本书普遍偏向推荐那些传统、通用、久经考验的工具。

深度理解

·核心概念：同一份“当前异常”信息，现代 Python 提供了

三种可选获取渠道：`except ... as X`（最直观）、`sys.excinfo()`（经典、三元组）、`sys.exception()`（3.11 新增、只给实例）。三者指向同一个底层对象——`sys.exception()` 与 `except ... as` 绑定的是完全相同的对象，`sys.excinfo()[2]` 则是它的"类型 + 实例"复刻。

·底层实现：`sys.exception()` 底层直接读"当前线程的异常槽位"（C 层的 `tstate->currentexception`），`sys.excinfo()` 返回 `(type, value, traceback)` 三元组。`call` 的最小实现是把已存储的异常实例取回；`exc` 则在 `handler` 结束后、异常对象可能被释放前就工作。若在 `except` 之外调用 `sys.exception()`，Python 3.11+ 返回 `None`（而不是报错）。

·为什么出现分歧：PEP 建议新 API 更简洁、倾向于直接返回异常实例；而以 Lutz 为代表的老派观点则认为：为一个一年前才新增的冗余调用，就堂而皇之把 `excinfo` 标成"old-style"，太过急进。这个"diss"（英文原节标题正是在 diss）恰好是语言演进与生态稳定之间张力的一次活教材。

·解决的现实问题：写日志、调试器、装饰器之类的通用代码时，只有一个异常实例而不想一路带 `type/traceback` 时，`sys.exception()` 让代码更简述——例如 `log = sys.exception()` 之后直接 `f'{log}'` 格式化。但它拿不到回溯对象，所以"完整诊断"仍然需要 `sys.excinfo()` 或 `e.traceback`。

·初学者容易：以为三者必须一样不可。实际上 `sys.excinfo()` 在 `handler` 执行结束后可能失效、返回的不再是当时

异常；`sys.exception()` 只能在"处理中"拿到；而 `as X` 绑定的变量在离开 `except` 块后会被解释器清空（3.x 自动 `del`）。同理，别在 `except` 之外调用 `sys.exception()`，不会得到理论值。

·点拨：本节正文其实暗含一条 Python 生态的"新风向"——3.11 起官方在逐步精简异常接口、减少"类型+值+追溯"这个传统三元组的暴露；值得注意的是 `logging.exception` 与 `traceback` 模块的代查仍基于 `excinfo` 三元组，兼容性无忧。

36.14 Displaying Errors and Tracebacks（显示错误与回溯信息）

英文原文

Finally, the exception traceback object available in the prior section's `sys.excinfo` data is also used by the standard library's `traceback` module to generate the standard error message and stack display manually. This module has a handful of interfaces that support wide customization, which we don't have space to cover usefully here, but the basics are simple. Consider the (some would say deservedly named) file in Example 36-5, "badly.py".

Example 36-5. `badly.py`

```
import traceback
def inverse(x): return 1 / x
try: inverse(0)
```

```
except Exception: traceback.printexc(file=open('badly.txt','w'))
```

print('Bye') This code uses the `printexc` convenience function in the `traceback` module, which uses the `sys.excinfo` data set (technically, in 3.11+ the implementation was aligned to read the same data via `sys.exception` instead). When run, the script prints the standard error message to a file—useful in programs that need to catch errors but still record them in full (again, type here is the Windows equivalent of Unix `cat`):

```
$ python3 badly.py Bye
```

```
$ cat badly.txt Traceback (most recent call last): File ".../LP6E/Chapter36/badly.py", line 7, in <module> inverse(0) File ".../LP6E/Chapter36/badly.py", line 4, in inverse
```

```
return 1 / x ~ ~ ^ ~ ~ ZeroDivisionError: division by zero
```

For more on `traceback` objects, the `traceback` module that works on them, and related topics, consult Python's standard-library manual and other references.

中文翻译

前一节中 `sys.excinfo` 数据里的那个异常对象，也正被标准库的 `traceback` 模块用来手动生成标准的错误信息与栈显示。这个模块有若干个支持广泛定制的接口——本书篇幅有限，此处不逐一展开——但基本用法很简单。来看看示例 36-5 中那个（“该被批评”的）名为 `badly.py` 的文件。

示例 36-5. `badly.py` `import traceback` `def inverse(x):` `return 1 / x` `try:` `inverse(0)` `except Exception:` `traceback.printexc(file=open('badly.txt','w'))` `print('Bye')` 这段代码使用了 `traceback` 模块里的 `printexc` 便捷函数，它内部取自 `sys.excinfo` 数据（技术上，3.11 起实现的读取路径已改经 `sys.exception` 的同一起来源）。运行时，脚本把标准错误信息打印到文件里，而不是屏幕上——这对“需要捕获错误、又想把完整记录保留下来”的程序很有用（再次说明，这里 `cat` 是 Unix 下查看文件内容命令，Windows 下对应的是 `type`）：
`$ python3 badly.py` `Bye` `$ cat badly.txt`
`Traceback (most recent call last):` `Line ".../LP6E/Chapter36/badly.py", line 7, in <module> inverse(0)` `Line ".../LP6E/Chapter36/badly.py", line 4, in inverse` `return 1 / x` `~~^~~` `ZeroDivisionError: division by zero` 关于回溯对象的更多内容、使用它们的 `traceback` 模块，以及相关话题，请查阅你手边的 Python 参考资料和文档。

深度理解

·核心概念：`traceback` 模块是“异常打印机”：把解释器默认的 `traceback` 输出“原样重放”并写到任意可写对象（文件、`sys.stderr`、`io.StringIO`.....）里。`printexc()` 是无参数的“就现在、写标准错误”快照，`formatexc()` 返回字符串版。

·底层实现：`traceback.printexc()` 内部先读“当前被处理异常”（旧实现 `sys.excinfo()`，3.11 后改由 `sys.exception()` 取得实例，再取实例的 `.traceback`——即回溯对象连第一章）。回溯对象是链表式的 `TracebackType`：每个

节点对应一帧 (tbframe、tblineno、tbnext)，遍历链就能把"文件名→行→源码行→错误"逐帧打印出来。

·为什么用文件而不是屏幕：生产环境的"静默失败"往往都要求把完整 traceback 写入磁盘 6小时的审计，之后运营团队再翻日志。这也正是 file= 形参存在的理由——打印方向可任意重定向。

·实际应用：自定义全局错误处理器 (sys.excepthook)、日志系统、远程错误上报、测试失败现场留证，都是 traceback 系列函数的常规战场。

·初学者误区：以为只有 except 里能打印回溯。其实用 traceback.formatexc()/printexc() 只需"正处于异常处理中"，否则得到"没异常"的空结果；另一个误区是 printexc() 把内容写向了 stdout——它默认写 sys.stderr，重定向要用 file=。

代码分析

```
import traceback

def inverse(x):
    return 1 / x

try:
    inverse(0)
except Exception:
    # "" SystemExit...
    traceback.print_exc(file=open('badly.txt', 'w')) #
    print('Bye')
```

·解释器如何处理：inverse(0) 执行 1 / 0 时，CPython 的整数除法检测到除零，抛 ZeroDivisionError；它被 except Exception 命中。此时解释器把"正在处理的异常"存

在线程级槽位里，`printexc` 从中取到异常实例，沿着 `traceback` 链把"第 7 行 `<module>` 调用 `inverse(0)` → 第 4 行 `return 1 / x` → 除法报错"逐帧格式化，写入 `badly.txt`。随后 `print('Bye')` 正常执行——因为异常已经被当初的 `except` 子句"消费"掉了。

·设计思想：`except Exception`: 在这里非常合适——它捕获"普通故障"但不碰 `SystemExit/KeyboardInterrupt`，让程序可以先"记账再继续"。这正是"捕获异常但保留现场"这一异常设计常见目标的挂标做法。

36.15 Exception Design Tips and Gotchas (异常设计提示与陷阱)

英文原文

This chapter is lumping design tips and gotchas together because it turns out that the most common exception gotchas stem from design issues. By and large, exceptions are easy to use in Python. The real art behind them is in deciding how specific or general your `except` clauses should be and how much code to wrap up in `try` statements. Let's address the latter of these choices first.

中文翻译

本章把"设计提示"与"陷阱"放在一起讲，是因为事实证明：最常见的异常陷阱都源自设计问题。总的来说，在 Python

中使用异常是很容易的；真正的艺术在于决定你的 `except` 子句应该多么具体或多么笼统，以及应该把多少代码裹进 `try` 语句。我们先处理后者。

深度理解

·核心概念：异常的两个"设计旋钮"——(1) `except` 的粒度 (specific vs general)；(2) `try` 块的边界（包多少代码）。本节是异常名副其实的"设计总纲"。注意作者用 `gotcha`（陷阱）来称呼它们：并非语法问题，而是会"阴"到你的设计失误。

·底层视角：这两个旋钮最终都反映到运行时行为——`except` 越宽，拦截面越大、越容易吞掉不属于自己的异常；`try` 包得越多，异常可能跨越的边界和清理链就越长。而它们都不改变语法正确性，错误只在"运行到那个分支"时显形。

·为什么重要：因为 Python 通常不会在语法期报设计错误——一个过宽的 `except` 本身很大概率跑得"很顺利"，只是把扩大 bug 藏在某条异常路径里。

·学习价值：这堆 `is`"跨章节遥相呼应"，所以正好用在本书核心部分的收尾，当作"出师的实战宣誓"。

代码分析（无新代码；以上设计取舍，适用于下文三个子主题）

36.15.1 What Should Be Wrapped (应该包裹什么)

英文原文

In principle, you could wrap every statement in your script in its own try, but that would just be silly (the try statements needed would then exceed the sequence itself!). What to wrap is really a design issue that goes beyond the language itself, and it will become more apparent with practice. But as a summary, here are a few rules of thumb:- Operations that commonly fail should generally be wrapped in try statements. For example, operations that need system state (file opens, socket calls, and the like) are prime candidates for try.- Unless they should fail... In a simple script, you may want failures to kill your program instead of being caught and ignored, especially if the failure is a showstopper.- Cleanup actions that must be run regardless of exception outcomes should generally be run with a try/finally combination, unless a context manager is available as a with option.- Wrapping the call to a function in a single try statement often makes for less code than wrapping operations in the function itself. That way, all exceptions in the function percolate up to the single try around the call.- The kinds of programs you write will probably influence the amount of exception handling you code as well. Servers, te

st runners, GUIs, and such, must generally catch and recover from exceptions. Simpler one-shot scripts, though, will often ignore exception handling completely, because failure at any step requires shutdown. In all cases, keep in mind that failures in Python normally generate useful error messages instead of hard crashes. Even without try, this is often a better outcome than some programs could hope for.

中文翻译

原则上，你可以把脚本里的每一条语句都包进各自的 try，但那只会显得荒谬（try 语句的数量会超过语句本身！）。该包什么实际上是超出语言本身的设计问题，随着实践会越来越明显。但作为总结，这里有几条经验法则（rules of thumb）可供参考：- 经常失败的操作为佳包进 try。例如与系统状态交互的操作（文件操作、socket 调用等）是 try 的首选候选。- 除非它们本就该失败——在简单脚本中，你可能希望故障直接杀掉程序，而不是被捕获后忽略，尤其是当故障是“卡点”（showstopper）级别的耗时。- 必须无论如何都要执行的清理动作，通常应该用 try/finally 组合来完成，除非有上下文管理器可以作为 with 方案。- 把对一个函数的调用包在单个 try 里，往往比把函数内部的语句逐个包裹代码更少。这样，函数里抛出的所有异常都会上浮到外层那一个 try 上。- 你写的程序类型也会影响异常处理代码的多少。服务端、测试运行器、GUI 等必须捕获并恢复异常；而更简单的“一次性脚本”则往往完全忽略异常处理——因为任何一步失败都意味着要“停机收场”。在所有情况下都要记得：在 Python 中，失败的通常会产生有用的错误信息

而不是硬崩溃。即便不用 `try`，这往往也比某些程序所能期望的结局更好。

深度理解

- 核心概念："把 `try` 用在刀刃上"——真正的失败风险聚集点（IO、网络、用户输入）需要兜底；而普通数学运算、常规流程则大可不必。

- 底层视角：CPython 3.11 起的异常处理是"零成本"的（`try` 块不在热路径上设障碍，只有真正抛异常才查异常表），所以"多用 `try` 包裹"的性能代价极小——主要的取舍在于逻辑的清晰度。

- 为什么适用"包一层调用而不是包内部逻辑"：包裹"函数调用"这一行，语义上等价于"我这个调用方的职责是为这整件事负责"；若包裹在函数内部的每一小步，将产生"负责粒度太小、维护时极易遗漏"的问题。这就是 "wrap the call" 的由来。

- 现实应用：Web 框架里最常见的写法正是 `try: handler(request) except: log`——业务函数内部不散落异常处理，统一在一个中间层收口。规则 4 因此是"大型框架"的骨架。

- 初学者误区：以为"要捕获异常就得把 `try` 包得紧紧"。结果包出一层又一层 `try` 嵌套，反而把"该由上层决定成败"的语义锁死。

36.15.2 Catching Too Much: Avoid Empty except and Exception (捕获得过多：避免空 except 与 Exception)

英文原文

As we've learned, Python lets us pick and choose which exceptions to catch, but it's important not to be too inclusive. For example, we've seen that an empty except clause catches every exception. That's an easy code and sometimes desirable, but it may also wind up intercepting an error that's expected by a try elsew here:

```
def func(): try:... # IndexError raised here
except:... # But everything comes and dies here!
try: func()
except IndexError: # Exception should be processed here...
```

Perhaps worse, such code might also catch unrelated critical events. Even things like memory errors, program typos, iteration stops, key presses (keyboard interrupts), and system exits raise exceptions in Python. Unless you're writing a debugger or a similar tool, such exceptions should not usually be intercepted in your code.

For example, scripts normally exit when control falls off the end of the top-level file, but Python also provides a built-in `sys.exit(status)` call to allow early terminations. This works by raising a built-in `SystemExit` exception to end the program, so that `try/finally` handlers run on the way out and tools can intercept the event. Because of this, a `try` with an empty `except` may unknowingly prevent an exit, as in Example 36-6.

Example 36-6. `exiter.py`

```
import sys
def bye(): sys.exit(62) # Aborts this program

try: bye()

except : # Oops—we've trapped the exit event
    print('Got it')
    print('Continuing...')
When run, the script happily keeps going after a call to shut it down:
```

```
$ python3 exiter.py
Got it
Continuing...
```

You simply might not expect all the kinds of exceptions that could occur during an operation. Using the built-in `Exception` superclass instead can help, because it is not a subclass of `SystemExit`:

```
try: bye()

except Exception: # Won't catch exits, but will catch many others...
```

In some cases, though, this scheme is no better than an empty `except` clause—in very tolerant nesting, the

re's no semantic difference at all. Worse, using either the empty except or Exception will also swallowing the very programming errors that should be kept visible. In fact, these two techniques can effectively turn off Python's error-reporting machinery, making it hard to notice mistakes in your code. Consider this code:

```
mydictionary = {...}...
```

```
try: x = myditctionary[key] # Oops: note the misspelling  
except Exception: x = None # Treated as "dictionary key missing" — wrong!
```

... The coder here assumes that the only error possible when indexing a dictionary is a missing-key error. But because the name `myditctionary` is misspelled, Python raises a `NameError` instead, and the empty exception handler will silently trap it, fill in `None`, and hide the mistake—all while the program keeps running. Diagnosing this later would be real interesting!

As a rule of thumb, be as specific in your handlers as you can be—empty except and Exception style catch—alls are convenient but can be error-prone. In the example, list `KeyError` rather than `Exception` or the empty clause. Better yet, structure your code so that any thing uncaught still propagates sensibly.

代码

```
def func():
    try:
        ...
        # IndexError
    except:
        # except
        ...
        # ""
try:
    func()
except IndexError:
    #
    ...
```

```
import sys
def bye():
    sys.exit(62)
try:
    bye()
except:
    # except SystemExit
    print('Got it')
print('Continuing...') #
```

中文翻译

正如我们已经学到，Python 让你可以自己挑选该捕获哪些异常，但注意不要过于包容。例如，我们已经看到空的 `except` 会捕获每一个异常。这容易写，有时也确实需要，但它也可能拦下被别处 `try` 所期望的那个异常：`def func(): try: ...` 此处抛出 `IndexError` `except:...` 但一切都会到这里，然后默默死在这里 `try: func() except IndexError: #` 异常本应在这里被处理...可能更糟：这样的代码还会捕获无关紧要的关键事件——内存错误、代码笔误 (typo)、迭代停止、中途按下 `Ctrl+C`、系统退出——这些在 Python 里都以异常的形式出现。除非你在写调试器之类的工具，否则这些一般不应在你的代码中被拦截。例如，脚本通常在顶层文件的控

制流走到末尾时会正常退出，但 Python 还提供了内置的 `sys.exit(statuscode)` 以支持提前终止。它通过抛出一个内置的 `SystemExit` 异常来结束程序，这样 `try/finally` 处理器会在退出路上运行、相关工具也能捕获这一事件。正因如此，带空 `except` 的 `try` 可能在无意中阻止了程序的退出——见示例 36-6。

示例 36-6. `exiter.py`

```
import sys
def bye():
    sys.exit(62) # 致命一个调用：立即终止程序
try:
    bye()
except:
    # 呵，我们把"系统退出"事件给吞了
    print('Got it')
    print('Continuing...')
# 运行它，脚本在一个本该关闭自己的调用之后依然快乐地继续执行（这正说明空 except 把 SystemExit 当成一般异常拦截了，本应立即退出的）：
$ python3 exiter.py
Got it
Continuing...
```

你可能根本意料不到一次操作会引发那么多种异常。改用内置的 `Exception` 超类会好一些，因为 `Exception` 并不是 `SystemExit` 的超类：`try: bye() # 抛出 SystemExit except Exception: # 不会捕捉 SystemExit`，但会捕捉许多其他异常...不过有些情况下，这方案与空 `except` 相比并不高明——在非常容忍嵌套的代码里，两者几乎无差别。更糟的是，空 `except` 或 `Exception` 二者之一都会一起吞掉那些本该暴露出来的编程错误（`typo` 之类）。事实上，这两招几乎可以关闭 Python 的错误报告机制，使你难以察觉自己代码里的错误。看这段代码：

```
mydictionary = {...}
try:
    x = myditctionary[key] # 糟糕：拼写写错了（mydictionary 少了字母 a）
except Exception:
    x = None # 误以为是"字典缺少键" ... 这里的编码者假设：索引一个字典只会发生"键缺失"这一种错误。但因为 myditctionary 拼错了，Python 抛出的是 NameError，而 except Exception 会把这次触碰收养，放进 None，然后继续运行，错误被彻底掩盖。以后再排查这个 bug
```

可有得乐了！作为经验法则：处理器尽量具体——空 `except` 与 `Exception` 这类"大网兜"方便，但可能是隐蔽的 bug。上面的例子，应该把 `except` 写 `KeyError`，而不是 `Exception`。更结构化一点，让没被处理的异常自然继续向上传播。

深度理解

·核心概念：异常设计的黄金法则之一是别兜底——Catch 至少 Broad，可 Handle at the right place。空 `except` 与 `Exception` 是"方便而危险"的两张网，会把不属于当前的异常（键盘中断、系统退出、甚至程序 bug 的 `NameError`）一并吞掉。

·底层实现：空 `except`：匹配 `BaseException` 基类的所有后代——包括 `SystemExit`、`KeyboardInterrupt` 这些"生死攸关"对象。`Exception` 则是"除上述事件外"的绝大多数异常总纲。两个的差别在实现上就是"继承树的哪一层"之差。

·设计原因：Python 用"退出即异常"的统一模型（`sys.exit` → `SystemExit`），好处是 `finally` 清理还能跑、调试器能拦截；代价就是——若谁用空 `except` 把 `SystemExit` 吞了，程序会"假装没事"继续跑，常会诱发更诡异的 bug。历史上这正是空 `except` 最经典的争议。

·实际建议：服务器长期任务、GUI 循环天然"需要恢复"，因而可以合理地宽捕获并记录日志；但临时脚本或模块则不必，让 `traceback` 替你喊叫。

·初学者误区：在"字典取键"这种只需 `KeyError` 的场景用

except Exception 甚至空 except, 等于把 name typo (拼写错误) 这类编程错误神秘。遇到"莫名返回 None、又找不到 bug", 第一个要怀疑的就是"宽 catch 掩盖了 NameError"。

设计类析 (产生上述两个代码片段时的洞察)

```
def safe(func, *pargs, **kargs):
    try:
        func(*pargs, **kargs)
    except Exception:
        #
        import sys
        import traceback
        print('FAILED:', sys.exc_info()[0], sys.exc_info()[1])
        traceback.print_exc(file=sys.stderr)
```

·设计加结合: 若你确实"只是想兜住普通模块级异常", 用 except Exception 而非空 except, 并叠加 traceback.print_exc 把"发生了什么"记录下来。还能把错误"升级"为 print, 而不是假设一切正常。

·注: 以上为练习 3 的临近预告——safe 函数正是取"所有期望内异常 + print + traceback", 是本节"不要太宽"的良善化应用。

36.15.3 Catching Too Little: Use Class-Based Categories (捕获太少: 使用基于类的分类)

英文原文

Being too restrictive in exception handlers can be just as troubling as being too general. When you list specific exceptions in a handler's `except`, you're catching only the ones you list. Any new exception that arises later must be added to every such handler in your system. We saw this phenomenon in the prior chapter. As a review, because the following handler is written to treat only `MyExcept1` and `MyExcept2`, a new `MyExcept3` introduced later in life still needs to be added to every handler list: `try: for ... except (MyExcept1, MyExcept2):...` (bug if `MyExcept3` is later thrown) Careful use of class-based exceptions can make this require fewer changes: catch a common superclass instead of listing each subclass — `try: ... except CommonCategory: # e.g., Error categories...` (future `MyExcept3` subclass included automatically) In other words, a little design upfront goes a long way to keeping exception-driven code flexible. The overall goal is to be neither too general (above) nor too specific (below): pick the just right category of exceptions, or use class hierarchies to organize "families" so later additions don't break existing handlers.

中文翻译

在异常处理器里做得过细，与做得过于宽泛一样令人头疼。当你在 `except` 里列出特具体的异常时，你捕获的只有你列

出的那些。系统以后新增异常类型，你就得一个个回到代码手册里把它加进每个处理器。上一章我们已经见过这种现象。回顾一下：因为这里的处理器只把 `MyExcept1` 和 `MyExcept2` 当成处理对象，以后系统再新增的 `MyExcept3` 就落空——`try: ... except (MyExcept1, MyExcept2):...`（若以后新增 `MyExcept3`，这里就漏接了如果善用基于类的异常分类，就能把这句「以后要改 20 处」的噩梦省掉多少：捕获一个公共的超类（superclass）而不是把所有子类都列出来——`try: ... except CategoryException: # e.g. 共同的父类...`换句话说，前端多想一步分类设计，能省下大笔维护工作。要点是既不能太宽（避免空 `except/Exception`），也不能太窄（只列一个个具体类）——选择在金的两批之间的“刚好”粒度：或者用继承把异常组织成了家族，让未来的新增类型天然地被祖类接住。

深度理解

- 核心概念：应提供的“异常三态”——过宽（empty/Exception）、过窄（只列具体类）、刚好（类分类的 superclass）。真正优雅的是用异常类树表达语义：同一个“业务域”的异常共享父类，处理的决定化就是判断“是否属于某个范畴”。
- 底层实现：`except SomeClass` 的匹配方式其实是“实例是 `SomeClass` 的实例吗”——这天然地支持子类继承：任何一个 `MyExcept3` 只要是 `CategoryException` 的后代，就会被 `except CategoryException` 接住。所以“分类”不只是文档层面的约定，直接被解释器的类型检查落实。
- 为什么这样设计：Python 的异常继承与类继承同构，语

义"警"直接可从异常类型体系读到：except 列表 = 你想接受的域，raise 时类型自然落进所属家族。这让异常树变成了一道"路线图"。

·现实平衡点：大多数团队实践是：-> 定义一个基础异常（如 `AppError`）；-> 各业务异常继承它；-> handler 里 `except AppError as e`：兜住一个域。这样恰好处在空 `except`（太宽）与逐条列举（太窄）中间。

·苗头判断：若你发现某个的 handler 里列出五六个异常元组 `except (A, B, C, D, E):`，这通常是"要开一个父类"的信号；若 `except Exception` 到处铺，则是"重构"最激烈的先兆。看一段代码的异常组织就能估出作者的架构功底，这也是面试官爱问的点。

36.16 Core Language Wrap-Up (核心语言收尾)

英文原文

Congratulations! So far you've completed a full loop of the core of the Python language. If you've gotten this far, you've got the essentials of the Python programming language. There's more optional reading in the Advanced Topics part ahead described in a moment. In terms of the essentials, though, the book's main journey—the core language—is now complete. Along the way, you've seen just about everything there is to see in the language itself—in enough depth to apply

to most code you're likely to meet in the Python "wild." You've studied built-in types, expressions and statements, and exceptions; you know how to build larger programs using functions, modules, and classes, plus your own extensions. You've also explored important software design issues, the full OOP paradigm, functional programming tools, program architecture concepts, and alternative tool trade-offs. That's a skill set you can now turn loose on the task of developing the essentials. Wait a moment—I should note that other programming languages have equivalents for these exception design tips, but their semantics differ widely. Use nuance and the Python style's emphasis on explicit, readable handling of rare events.

中文翻译

恭喜！至此，你已经把我们书库中 Python 核心语言的关键旅程走过一遍。到这里，你已经具备了 Python 语言的基础（基本）能力。前面还有“高级主题”部分的选读内容，稍后介绍。但就核心而言，这本书的主线旅程——这极语言核心——到此已经完成。一路走来，你已经见过语言本身的几乎一切——并且深入足以应付你在 Python“野生代码”中可能碰到的大多数代码。你已经研究了内置类型、表达式、语句、函数、模块、类，并且知道如何用它们来构建更大的程序。你还探讨了软件设计问题、完整的面向对象范式、函数式工具、程序架构概念、工具取舍——这一技能套装现在已经可以放教开发真实的应用程序了。或许也应该指出，这些异常设计概念在其他语言中也有对应物，但它们的实现

与观点相当分化。请保持思辨，并结合 Python 对"显式优于隐式"的强调，来让异常真正serving。

深度理解

·核心概念：这是"核心语言课程的毕业典礼"。第 1 章承诺的动态类型、字节码、OOP、可读性，都已在 15~36 章的漫游中兑现完。

·为什么把"异常设计"单独立项在这个位置：异常是"语言功能手册"的最后一个主题——接下要离开手册进入"应用开发工具"，所以它最值得"按惯例固化出原理"。

·额外视角：Lutz 的"words→设计"节奏，从第 1 视角一路下来的"代码即知识的风格"，至此版一套"可以独立写库的权限"，这就是本章"毕业"著作。

36.16.1 The Python Toolset (Python 工具组)

英文原文

From this point forward, your Python career will largely consist of becoming proficient with the toolset available for application-level Python programming. You'll find this an ongoing task. The standard library, for instance, contains hundreds of modules, and the public domain offers still more tools; it's possible to spend years gaining proficiency with all of them. Speaking generally, Python provides a hierarchy of tool tiers:-

Built-in tools: Built-in types like strings, lists, and dictionaries make it easy to write simple programs quickly. - Python-coded extensions: For more demanding tasks, you can write your own functions, modules, and classes. - Extensions in other languages: Although not covered in this book, Python can also be extended with code written in C, C++, or Java. Because Python uses layered toolkits, you decide how deep to go for any given task: use built-ins for simple scripts, add Python-coded extensions for bigger systems, and code other-language extensions for heavy lift.

中文翻译

从这以后，你的 Python 职业生涯主要就是不断熟悉那套“应用级 Python 编程”可用的工具集。你会发现这是一项持续的性质。标准库就有数百个模块，公共领域还有更多。花多年逐完成大家，简直可能。广义上说，Python 提供一层工具：工具分层体系： - 内置工具（Built-in tools）：字符串、列表、字典等内置类型，让简单程序飞速上手。 - Python 编写的扩展：对更大任务，你可以自行编写自己的函数、模块、类。 - 其他语言扩展：还可以用 C、C++ 或 Java 编写扩展（本书不展开）。由于 Python 是分层工具（tool kit layering），你可以按任务需要决定往多层下钻：简单脚本用内置，比较大系统加 Python 扩展，高要求系统再搞外部扩展。

深度理解

·核心概念：Python 的可用性取决于“三层水库”——内置

、自写扩展、外部语言扩展。多数实际系统其实是内置 + Python 扩展的组合。

·与 C 的对比：C 往往只有“编译进可执行”一层；python 从 REPL 到 C 扩展，处处可行。所以“缩小需要啊”的项目规模划分是学习重点。

36.16.2 Development Tools for Larger Projects (更大型项目的开发工具)

英文原文

Most of the examples in this book have been fairly small and self-contained. They were written that way on purpose, to help you master the basics. But now that you know all about the core language, it's time to start learning how to use Python's built-in and third-party interfaces to do real work. The :brief this section is a quick tour of the tools in this domain: documentation tools (PyDoc's help and HTML interfaces), error-checking tools (PyChecker, Pylint, Pyflakes), testing tools (PyUnit/unittest, doctest), IDEs (IDLE, PyCharm, Jupyter notebooks, etc.), profilers (the profile module and its optimized relative cProfile, plus pstats), debuggers (the standard pdb module), shipping options (standalone executables, .py, .pyc, .zip, and the pip installer), optimization options (PyPy, Shed Skyar/you Cycle), and installation/virtual-venv management.

See also: "Stacking", Python especially for larger projects: 模块包 (第24章)、异常类设计 (第34章)、伪私有类属性 (第31章)、文档字符串 (第15章)、模块数据隐藏 (第25章)。

中文翻译

本书中的大多数例子都比较小、自成一体。这是特意如此安排的，旨在帮助你掌握基础。但现在你已经吃透核心语言，该学用 Python 的内置与第三方接口去做真正的工作了。本节微信号的快速导览，围观这个领域里的工具族：- 文档工具 (Documentation tools)：PyDoc 的 help 函数与 HTML 接口 (在例子的第十五章讲的)，整合 docstring。- 错误检查工具 (Error-checking tools)：PyChecker、Py lint、Pyflakes ——对标 C 语言的 lint，能在运行前拦下常见错误。- 测试工具 (Testing tools)：PyUnit (标准库里叫 unittest)，剪 JUnit 的对象式框架；doctest——把交互式示例贴进 docstring 做回归。- IDE：IDLE、PyCharm 等提供图形化编辑/运行/调试；Jupyter notebooks 也可视作一种 IDE。- 剖析工具 (Profilers)：profile/cProfile 性能剖析，pstats 分析结果；"先写清楚，再剖析，再优化"。- 调试器 (Debuggers)：标准库 pdb，命令行式、类似 C 的 gdb；大多 IDE 自带 GUI 调试。薄不少实践中常见还是 print 打点。- 发布选项：源码 .py、字节码 .pyc、.zip 包、独立可执行、pip 安装器。- 优化选项：PyPy 加速、Shed Sky (研究)、Cython；或把关键部分用 C 改写。- 环境管理：虚拟环境 venv，即多套 Python 包隔离安装。

深度理解

- 核心概念：本书唯一一次列出"开发工具箱"的地方——说明作者的判断是：学到 `www` 脚本之后，这些工具的"生态位"开始变得重要。
- 真实优先级：这些工具到现在 2026 年都已演化多年：`unittest` 仍是主流标准测试框架，`doctest` 偏轻量；`PyPy/Cython` 路线成熟；`venv` 已是默认。本节的价值=按图索骥。
- 开发定位：尽管文本当初写于"可以不用"时代，但开发大型系统时，文档（`docstring`）、测试（`unittest`）、静态检查（`pylint/mypy`）已经是行业默认三件套。时刻自问："我读代码时能像读自己刚写的吗"。
- 本章语义：这一节实质是把"语言出师清单"设置为30星级的"工具至今"。

36.17 Chapter Summary + Quiz (章节小结与测验)

This chapter wrapped up the exceptions part of this book with case reprs, and it wrapped up the book's core-language material. If you've reached these words, you're likely now ready to be called — to use the author's phrase—a fully-fledged Python programmer. The next and final part of this book is a collection of optional chapters on advanced "core-language" topics. They're optional or deferrable, since not every Pyth

on programmer needs all of them, and many businesses can stop here and start exploring Python's real application roles. Indeed, in practice, application coding is often more important than advanced (and sometimes exoteric) language features. If you do feel the very next steps are the more advanced subjects, the final part's chapters will help you get started on topics such as Unicode, binary, objects with descriptors/decorators/metaclasses, and larger example applications. Given the special place of this chapter (closing the core), the chapter quiz has just one question.

Test Your Knowledge: Answer (一个问题的测验与答案)

Quiz

Q: What's up with the mouse (???) on the cover of this book?

Answer

Not actually a mouse? A wood under rat, actually — from the family *Neotoma*. Wait: Actually the mammal on the cover—named a wood rat—was chosen by the book's publisher. The electronic — Yes: a real ? — and clever : "住参考" — Let's be honest: The "wood rat"/pack-rat was taken up as a mascot because: 1) wood rat loves things that shine/collecting (收集爱好) ; 2) python foods eat the wood python. Nice tie-in — but

also a wink: learning Python protects you from being "消化" by errors (?), 反正没有 coroutine, abundance. For officially confirming: the first edition's jacket design, the pun "rat with the python%" ...

Test Your Knowledge: Quiz

1. What's up with the mouse on the cover of this book?

Test Your Knowledge: Answers

1. The duct—read "guinea... without reading" answer:

OK, this was never said in the book, but the mouse is actually a wood rat (*Neotoma muridae*)—selected by the publisher for same "it's a kind of python's prey." The 'rat must learn about the python or get eaten' metaphor. It also references Neotoma's behavior: pack rats shine to collect whatever they find — as an apt metaphor for Python's accretion of features over the years. Enjoy the shiny things, but try not to get eaten by constricting reptiles in the process.

Test Your Knowledge: Part VII Exercises (第 VI I 部分练习)

Since we've reached the end of the book's value, it's time for a few exception exercises to make it practice

the basics. The solutions are in appendix B under "Part VII, Exceptions".

1. try/except: Write a function called `oops` that explicitly raises an `IndexError`. Write another function that calls `oops` inside a try/except to catch it. What happens if you change `oops` to raise a `KeyError` instead? What do `KeyError` and `IndexError` come from? (Hint: won all unqualified names come from one of the four scopes?)

2. Exception objects and exception classes: Change `oops` to raise an exception you define yourself, called `MyError`. Identify your exception and its class with a class of your own. Then extend the handler in the catcher to catch `IndexError` and also this new class; print the "instance" you catch as an exception object.

3. Error Handling: Write a function `safe(func, pargs, kargs)` that runs any function with any number of positional and keyword arguments (use the arbitrary arguments header), catches any exception raised during the run, and prints the exception using the `excinfo` call in the `sys` module. Then use `safe` to run your `oops`. Put `safe` in a file: `exctools.py`, and pass it `oops` interactively. What error messages do you get? Finally, expand `safe` to also print a Python stack trace when an error occurs, by calling `printexc` in the standard-library `traceback` module (see earlier in this chapter).

4. Self-study examples: At the end of Appendix B, the book lists a fe

w example scripts developed as in-class group exercises, for you to study alone with the manual. They're not described, but they show how the concepts come together in real programs. If those apt ping your interest, find larger applications in follow-up books and on the web.

本章总结

第 36 章是全书的"核心语言毕业礼": 它不教新语法, 而是把异常设计的艺术(嵌套路由、惯用法、设计粒度)与开发者工具箱一次性交付。以下总结站在"读完本章、即将做应用开发"的位置上, 梳理你需要带走的東西。

技术拓展 (Technical Expansion)

- 实际项目中的应用场景:
- 日志与诊断: `logging.exception()` (内部基于 `excinfo` 三元组) 是生产环境记录"带完整栈"异常的标准方式; 本章的 `traceback.printexc(file=...)` 与 `formatexc()` 则是把诊断写到文件、字符串或上报服务的底层零件。记住: 生产代码里的 `except` 应当"记录后重抛或继续", 而不是 `print` 一句就完事。
- 错误边界 (error boundary): Web 框架 (如 Django 中间件、Flask `errorhandler`) 与 GUI/客户端程序都会在"程序外壳"包一层外层 `try`, 把未捕获异常统一转成日志

+ 用户可读的 500 页/错误弹窗——这正是 36.10 节"外层 try 调试"的工业化版本。

·重试 (retry) : 网络请求、数据库连接的偶发失败常用"捕获特定异常 (如 `ConnectionError`、`TimeoutError`) → 指数退避重试 → 若干次后失败"的模板。关键设计点: 只重试"可恢复"的异常子集, 别把 `KeyboardInterrupt`/编程错误也重试了——这正是"异常分类" (36.15.3) 的实际价值。

·用户自定义异常: 为业务域定义异常族 (如 `PaymentError`、`OrderNotFoundError` 继承自 `AppError`) , 用父类 `except AppError` 一处兜住整个域; 配合 `raise ... from` 保留根因, 是三层架构 (ORM/服务层/接口层) 的标准做法。

·异常组 (exception group) : `asyncio` 并发任务批量失败、校验器一次性收集所有字段错误时, `ExceptionGroup + except` (3.11+) 让"一批错误一起传播、按类型分发处理"成为可能——但要克制, 单异常 + 类层次能解决的场景不需要组。

·信号式异常: 把 `EOFError`/`SystemExit`/`KeyboardInterrupt` 当控制流信号 (36.7 节) 的思维, 正是 `asyncio.CancelledError`、`StopIteration` 这类"非错误异常"的设计原型——异常是通用的"跨层通知机制"。

·与其他语言 (Java/C++) 的区别:

维度	Python	Java	C++	
----	--------	------	-----	--

异常基类	BaseException 下分 Exception (业务) 与 SystemExit/KeyboardInterrupt (信	统一 Throwable (Exception/Error 两级)	任意类型均可 throw (惯例上抛类实例)	
必须声明吗	否——无 checked exception, 调用方自行决定是否捕获	是——checked exception 必须在签名 throws 声明	否, 但实现未定义行为风险	
检查异常	无 (全靠运行时)	编译期强制检查	无	
多重捕获	except (A, B) 元组或 except (异	catch (A \	B e)	catch(...) 按类型匹配
资源清理	with 上下文管理器 (协议化)	try-with-resources (Java 7+)	RAII: 析构函数, 异常无关自动释	
异常链	raise ... from (PEP 3134, 3.0+)	构造函数原因链	无内置支持	
裸重抛	raise	throw;	throw;	
性能模型	3.11+ 零成本异常 (不抛时几乎无开销)	JIT 优化后开销低	不抛异常时通常零开销 (取决于 ABI)	

Python 最大的哲学差异是没有 checked exception: 异常约定靠文档与纪律维护, 代价是"宽捕获"容易掩盖 bug (36.15.2 的拼写错误示例), 收益是函数签名不被异常声明污染、高层代码写起来自由。C++ 的 RAII 把清理绑定到对象生命周期, 而 Python 选择 with/finally 的显式协议——两者殊途同归, 但 Python 把"何时清理"的决策权留给了程序员。

· Python 发展历史背景 (务必以史为鉴、严谨不编造) :

- 类异常取代字符串异常：Python 2.6 之前异常可以是任意字符串；PEP 352 (Python 3.0) 强制所有异常继承 `BaseException`，并重构异常层级 (`SystemExit/KeyboardInterrupt` 与 `Exception` 分家)，才有了本章“空 `except` 吞掉退出事件”的讨论基础。
- PEP 343 (2005, Python 2.5)：引入 `with` 语句与上下文管理器协议 (灵感来自 C# 的 `using`)，从根上减少了 `try/finally` 样板。
- PEP 3134 (2005, Python 3.0)：引入 `context/cause` 异常链与 `raise ... from; except Exception as e` 中 `as` 变量的“退出即删除”策略 (PEP 3110) 也是同时期确立，为打破 `traceback` 循环引用。
- PEP 463 (2014) 未通过：曾有人提议“异常捕获表达式” (`expr except Exception: default`，试图让 `try/except` 成为可内联进表达式的一行写法，类似 `dict.get`)。该提案因“收益有限、与现有语句式写法重叠、可读性争议”而被否决——这是 Python 设计哲学“宁可语句显式，不搞表达式魔法”的典型案列，也解释了为什么至今没有 `try` 表达式。
- PEP 654 (2022, Python 3.11)：引入 `ExceptionGroup` 与 `except`，为 `asyncio` 等并发场景的“多异常同时传播”提供了一等支持；`sys.exception()` 也是 3.11 新增 (本章 36.13 节详述)。
- 零成本异常与增强回溯：PEP 657 (3.11) 给出精确异常位置 (`~~^~~` 标注)，配合异常表跳转实现“不抛异常时 `try` 零开销”——这让“多用 `try` 包代码”的性能顾虑彻底放

下。

·高级开发者需要掌握的相关知识：

- `traceback` 模块：`printexc/formatexc/formatexception/extracttb` 的差异与使用场景；自定义 `sys.excepthook` 全局接管未捕获异常（记录到文件、发送告警）。
- `logging` 模块：`logger.exception()`、`excinfo=True` 参数、`LOGLEVEL` 与异常分级——生产级错误观测的正确姿势。
- `contextlib` 家族：`@contextmanager`（生成器形态的上下文管理器）、`ExitStack`（动态入栈多个清理动作）、`suppress`（声明式忽略指定异常）、`redirectstderr` 等——“资源管理”的终极工具箱。
- 异常对象内部结构：`args`、`traceback`、`withtraceback()`、`cause/context/suppresscontext`，以及自定义异常类重写 `__str__` 的惯例（见第 35 章）。
- `sys.excinfo()` 的线程语义：每个线程有自己的“当前异常”槽位，跨线程传异常需显式传递异常对象本身。

学习建议 (Learning Advice)

- 重要程度：★★★★★ (5/5) ——异常设计是 Python 程序员从“会语法”到“会写生产代码”的分水岭；本章的嵌套路由、`except` 粒度、`with` 取舍直接决定你写的程序在出错时是“优雅降级”还是“雪崩+丢现场”。
- 应该掌握到什么程度：

1. 必须：能画出"嵌套 try/except 与 try/finally 在异常时的路由图"（最近匹配者优先 vs. 沿途全部执行）；能解释"语法嵌套 $\equiv$ 运行时嵌套"。

2. 必须：三条惯用法脱口而出——raise 多级跳出循环（36.6）、EOFError 当信号（36.7）、raise 传状态替代哨兵值（36.8）；并能在实际代码里识别它们的变体。

3. 必须：with 与 try/finally 的取舍判断（有上下文管理器用 with，否则 finally）；知道 sys.excinfo() / sys.exc_info() / as X 三者关系（36.12–36.13）。

4. 必须：设计粒度三原则——不空 except、不滥用 Exception、用类层级组织"异常家族"（36.15）；能讲出"宽捕获掩盖 NameError"的经典反例。

5. 熟练：用 traceback/logging 把异常完整落盘；了解异常组 except 的适用边界。

6. 理解：开发期"外层 try 兜底"与生产期"fail fast + 监控"的分界线（36.10 的陷阱）。

·后续应该学习的内容：

·第 35 章（异常对象）：若本章是"怎么设计异常"，第 35 章就是"异常类从哪里来"——继承树、except 语法细节，两章合起来才是完整图景。

·第 37–39 章（字符串、文件、面向对象）：下一部分的高级主题中，文件/二进制处理需要扎实的"资源清理"直觉（with 处处出现）；OOP 章节（类、装饰器、元类）会把"自定义异常类 = 普通类继承"这一事实放大成元编程玩法——第 35/36 章打的地基正是理解元类的入口。

- 练习落地：完成本章"Test Your Knowledge: Part VII Exercises"的 4 道题，尤其是 safe 函数（pargs/kargs + excinfo + printexc 三合一），它是"装饰器时代"的预热。

- 进阶阅读：contextlib 文档、logging HOWTO、traceback 模块文档；真正动手写一个带重试与错误上报的小型请求封装，把本章所有设计点串起来。